

Research article



Blockchain-assisted attribute-based multi-keyword search for dynamic encrypted data in cloud-edge-IoT

Hanlei Cheng ^a, Sio-Long Lo ^{b,*}, Jing Lu ^{c,d}

^a College of Computer Science and Cyber Security, Chengdu University of Technology, Chengdu 610059, China

^b School of Computer Science and Engineering, Macau University of Science and Technology, Macau 999078, China

^c College of Computer Science and Artificial Intelligence, Hubei University of Education, Wuhan 430205, China

^d Blockchain and Metaverse Laboratory, YGSoft Inc, Zhuhai 519085, China

ARTICLE INFO

Keywords:

Attribute-based keyword search

Data sharing

Blockchain

Search verifiability

Revocation and traceability

ABSTRACT

Keyword search is a fundamental technique for retrieving data outsourced to the cloud. Although encryption preserves data confidentiality, existing searchable encryption schemes often fail to efficiently support dynamic authorization and flexible retrieval. To address these limitations, we propose BAMKS, a blockchain-assisted attribute-based multi-keyword search scheme that supports secure and efficient search over version-aware encrypted data. In BAMKS, multiple data owners collaboratively generate version-bound access tokens that grant authorized users decryption privileges over evolving data. The scheme further enables conjunctive keyword search with updatable indexes. To ensure the integrity of search results, users can verify their correctness using an aggregated Schnorr-based non-interactive zero-knowledge proof, which is validated by smart contracts. In addition, BAMKS provides efficient attribute and user revocation without re-encrypting the stored ciphertexts, and supports user traceability for identifying malicious users from leaked keys. We formally prove that BAMKS achieves security against chosen-plaintext attacks (IND-CPA) and chosen-keyword attacks (IND-CKA) under the Decisional Bilinear Diffie-Hellman (DBDH) assumption. Performance evaluations show that the scheme achieves lightweight decryption and efficient multi-keyword search, thereby reducing client-side computation and making it suitable for resource-constrained IoT environments. These features demonstrate the practicality of BAMKS for distributed cloud-edge-IoT storage applications.

1. Introduction

With the increasing convergence of cloud-edge computing and the Internet of Things (IoT), more end-users are outsourcing personal data from their resource-constrained IoT devices to scalable, pay-as-you-go cloud storage providers (CSPs). This outsourcing process effectively alleviates the local storage burden. Edge servers act as a bridge between the centralized cloud and distributed IoT devices. They are equipped with cloud-like computing resources, enabling them to cache data to serve IoT users with low latency and high data processing capability. This cloud-edge-IoT paradigm facilitates data sharing between IoT end-users and business providers while introducing various security and privacy challenges in data access and retrieval [1]. For instance, patients' sensitive electronic healthcare records (EHRs) could be leaked to the semi-trusted cloud or edge servers [2]. To solve this security issue, it is conventional

* Corresponding author.

E-mail addresses: chenghanlei@cdut.edu.cn (H. Cheng), sllo@must.edu.mo (S.-L. Lo), lujing@hue.edu.cn (J. Lu).

<https://doi.org/10.1016/j.iot.2025.101838>

Received 17 September 2025; Received in revised form 9 November 2025; Accepted 29 November 2025

Available online 6 December 2025

2542-6605/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

to encrypt the owners' data before outsourcing them to the cloud [3]. However, it is necessary to consider how to securely search keywords to retrieve the target encrypted while preserving the confidentiality of outsourced data.

Attribute-based encryption (ABE) [4] and searchable encryption (SE) [5] have emerged as foundational techniques for attribute-based keyword search (ABKS) [6], which serves as a promising approach that authorizes legitimate users to search encrypted data using specific keywords [7]. In ABKS schemes, in addition to encrypting the data based on the ABE algorithm, each keyword in the keyword dictionary is encrypted under an access policy to generate a corresponding index. In contrast, a query keyword is encrypted with a user attribute set to produce a trapdoor. Access is granted only if the user's attributes satisfy the access structure [8] and the encrypted keyword index textually matches the query trapdoor [9].

Existing ABKS schemes [10,11] mainly focus on keyword search over static datasets and lack effective mechanisms for handling dynamic updates. In fact, in many real-world applications (e.g., remote email services or electronic health record (EHR) systems), outsourced documents evolve continuously, requiring synchronous adjustments to both users' decryption privileges and the associated encrypted keyword indexes. In such streaming scenarios, continuous data updates form a data stream, which necessitates a continuous access and query mechanism [12] to ensure secure and accurate retrieval over the evolving ciphertexts.

Furthermore, most state-of-the-art ABKS schemes enable *one-to-many*¹ data sharing [13]. However, in distributed IoT-based applications, data users often require access to data from multiple sources in cloud environments, leading to *many-to-one* or *many-to-many* data sharing demands [14]. In addition, most ABKS schemes are limited to single-keyword searches, which not only reduces search accuracy but also leads to inefficient use of computational resources.

1.1. Motivation

These limitations reveal several key challenges that motivate our work:

- In dynamic data sharing involving multiple data owners, users' decryption privileges need to evolve consistently with the ongoing data updates. However, most existing ABKS schemes do not provide an efficient mechanism for jointly authorized key management.
- As outsourced datasets continuously grow, their encrypted keyword indexes are expected to support scalable conjunctive keyword search while preventing leakage of search or access patterns.
- Beyond secure retrieval, users should be able to verify the correctness of returned search results without exposing sensitive query or plaintext.
- Current ABKS schemes generally rely on re-encrypting stored ciphertexts during attribute or user revocation, which incurs additional computational overhead and is impractical in resource-constrained environments.

To address these challenges, we construct a **B**lockchain-assisted **A**tttribute-based **M**ulti-**K**eyword **S**earch (BAMKS) scheme over dynamic encrypted data, which supports secure and efficient data sharing.

1.2. Contributions

Our scheme makes the following main contributions:

- **Efficient sharing of dynamic files in multi-owner settings.** We propose BAMKS, which enables data owners to jointly generate version-bound access tokens. The token is protected through a hybrid off-chain/on-chain ABE mechanism, providing efficient and decentralized access control for dynamically updated files.
- **Secure conjunctive multi-keyword search with index updates.** Our scheme allows users to conjunctively query multiple keywords when creating a search trapdoor, thereby enabling efficient and secure retrieval of target files. In addition, to enhance search scalability, our solution supports updating the keyword indexes of encrypted files.
- **Anonymous and verifiable search results.** We extend a Schnorr-based non-interactive zero-knowledge (SNIZK) proof to construct an aggregated SNIZK proof for matched files. The correctness of the retrieval results is verified by smart contracts without revealing the queried keywords or file contents.
- **Revocation and user traceability.** BAMKS supports both attribute and user revocation without requiring the re-encryption of all stored ciphertexts. Moreover, a built-in traceability mechanism allows identifying misbehaving users from leaked keys, ensuring accountability in practical deployments.

The remainder of this paper is organized as follows. [Section 2](#) reviews the related work. [Section 3](#) introduces the preliminaries of the proposed BAMKS scheme. [Section 4](#) defines the problem formulations, and [Section 5](#) presents the detailed construction of BAMKS. The correctness and security analysis is presented in [Section 6](#), and the performance evaluation is given in [Section 7](#). Finally, [Section 8](#) concludes the paper.

¹ In the context of data sharing, "one-to-many" means that multiple users can access data from a single owner, provided that their attributes satisfy the access policy.

2. Related work

2.1. ABKS

Attribute-based keyword search (ABKS) inherits from attribute-based encryption (ABE) [4,15] and searchable encryption (SE) [5]. Zheng et al. [6] were the first to propose the cryptographic primitive of verifiable attribute-based keyword search (VABKS), which allows users who satisfy the access structure to search over encrypted data while verifying the correctness of the search results.

ABKS has attracted considerable attention in both academia and industry. Varri et al. [16] developed FELT-ABKS, a fog-enabled lightweight and traceable ABKS scheme that supports multi-keyword search and decryption for authorized users, while enabling both user and attribute revocation. To further trace users who misuse their decryption keys, the authors subsequently proposed TL-ABKS, which employs white-box traceability to identify malicious users [17]. However, these schemes primarily focus on static data access in single-owner settings and do not address dynamic authorization and retrieval over continuously evolving datasets. Shi et al. [18] proposed an ABKS scheme for information sharing in social applications. Their scheme allows users to search over encrypted data based on attribute policies; however, it only supports single-keyword queries, limiting practical usability. Similarly, Yin et al. [19] introduced PH-ABKS-DS, a policy-hiding ABKS scheme based on prime order groups. While it enhances privacy by hiding access policies during search, it also restricts queries to a single keyword. In the Social Internet of Vehicles (SIOVs), Gu et al. [20] designed a traceable ABKS scheme in which data-owning vehicles partition and upload encrypted data blocks with associated keywords, while data-requesting vehicles generate trapdoors to retrieve required data. Although this approach supports traceability, it still relies on single-keyword queries and does not efficiently handle large datasets, often returning many irrelevant results [10].

To improve retrieval efficiency and flexibility, several works have explored multi-keyword search techniques. Zhang et al. [21] developed an ABKS system that supports conjunctive keyword queries and designates an independent auditor to verify search results. Liu et al. [22] proposed EMK-ABSE, in which the data owner partially encrypts both data and associated keywords online, enabling edge nodes to perform the search process and thus improving search efficiency. Huang et al. [23] introduced ABERKS, which supports expressive Boolean queries using AND, OR, and NOT operators, and ranks matched files based on relevance. Li et al. [24] proposed a tree-based structure for conjunctive keyword searches, enhancing query efficiency in multi-keyword scenarios. Chen et al. [25] presented PHP-ABMKSE-VR, a partially hidden-policy ABKS supporting multi-keyword search, verification, and revocation. Their scheme also resists attribute-value guessing attacks. Despite these advancements, these schemes are mostly designed for data sharing in single-owner settings, which leads to redundant authorization overhead once multiple data owners are involved.

Motivated by collaborative data sharing, recent studies have proposed multi-owner ABKS schemes. Wang et al. [26] introduced AESM², supporting multi-owner, multi-user access with a single trapdoor and efficient owner-level revocation without updating ciphertexts. Yin et al. [27] designed an ABKS scheme for cloud-assisted IIoT environments, where a novel master key generation and private key aggregation mechanism allows all data owners to collaboratively generate a user's private key using only their respective master keys, simplifying key management. In the Internet of Vehicles (IoV), Li et al. [28] proposed FAMI, a fuzzy ABKS scheme supporting many-to-many access control. FAMI employs an indistinguishable Bloom filter (IBF) and bigram keyword conversion to improve the accuracy of similarity matching for multi-keyword searches.

These studies advance ABKS; however, they primarily focus on static datasets. Very few ABKS schemes simultaneously support dynamic data sharing in multi-owner settings, multi-keyword search, verifiable retrieval, efficient revocation, and user tracing.

2.2. Blockchain-based ABKS

Benefiting from the intrinsic properties of blockchain, such as consensus, transparency, tamper resistance, and non-repudiation [29], blockchain-based ABKS schemes have been extensively explored [30,31]. Existing works typically address one or several specific functionalities, such as multi-keyword retrieval, multi-authority settings, hidden policies, malicious user tracing, or user revocation.

For example, Xiang and Zhao [32] developed a blockchain-based searchable attribute-based encryption (SABE) scheme for health-care applications, while Su et al. [33] introduced BA-RMKABSE, a blockchain-aided ranked multi-keyword searchable encryption scheme with hidden policies, where smart contracts [34] are employed to match trapdoors with keyword indexes and verify search results. Zhang et al. [35] proposed a verifiable and fair attribute-based conjunctive keyword search (ABCKS) scheme that leverages blockchain as a decentralized, tamper-proof index repository to ensure trustworthy verification. Wang et al. [36] presented an ABSE-based knowledge sharing architecture for intelligent IoT, which integrates a permissioned blockchain with a reputation-enhanced Delegated Proof of Stake (DPoS) protocol to maintain decentralized consensus among edge nodes. Zhang et al. [37] further proposed a BADS scheme that enables DOs to encrypt keyword indexes using an anonymous CP-ABSE algorithm and store the resulting cipher indexes on a consortium blockchain.

To mitigate the risks of master key abuse, Yin et al. [38] introduced a DCP-ABSE scheme in which both the system master key and user secret keys are jointly generated by attribute authorities acting as blockchain nodes. Shen et al. [39] proposed BMSE, which applies Simhash, *K-means*, and TF-IDF to cluster electronic medical records (EMRs) and return the top-*k* results. Yu et al. [40] developed BC-dPEKS, enabling multi-keyword search with trapdoor generation via the Pedersen Protocol on a permissioned blockchain. Alamer et al. [9] developed the security and privacy blockchain-based data sharing (SPB-DS) system, in which requesters create trapdoors to match keywords on the blockchain, and a payment mechanism is designed to incentivize stakeholder participation. Yan and Zhang [41] proposed an ABSE scheme for IoT that employs an attribute Bloom filter for access policy protection and query prefiltering, while utilizing blockchain for index storage, trapdoor management, and preliminary filtering. Hou et al. [42] designed

BC-RMABSE for IoV, which combines the vector space model with Pedersen's (k, n) secret sharing protocol on consortium blockchain to support ranked multi-keyword search and address the single-point-of-failure issue.

Despite these advancements, most blockchain-based ABKS schemes primarily exploit blockchain as an immutable storage for encrypted data or keyword indexes. They are limited to static data scenarios that do not support dynamic updates. Therefore, there remains an urgent need for blockchain-based ABKS schemes that not only enable dynamic data sharing in multi-owner settings, but also provide secure and efficient keyword search capabilities.

3. Preliminaries

3.1. Blockchain

Blockchain, initially popularized by Bitcoin [43], is a distributed ledger technology that maintains transactions in a chronologically append-only ledger replicated across a network of nodes. Each block contains a set of transactions together with the cryptographic hash of the previous block, thereby providing integrity and tamper-resistance through primitives such as hash chaining and digital signatures [44]. With its decentralized and immutable architecture, blockchain offers a trustworthy foundation for secure data sharing in cloud-edge-IoT scenarios [45]. Moreover, blockchain platforms support smart contracts, i.e., self-executing programs deployed on the ledger that enforce predefined rules without centralized authority [46]. Smart contracts enable transparent and automated decryption privilege verification and access policy enforcement [47]. When combined with access control models (e.g., ABAC), blockchain can record authorization activities and operation logs on-chain, thereby addressing traceability and accountability limitations in traditional centralized systems. These hybrid approaches provide consistent, auditable, and decentralized access control [48].

3.2. Access structure

An access structure $\mathbb{A}$ over an attribute universe $\mathcal{U}$ is a family of non-empty subsets, i.e., $\mathbb{A} \subseteq 2^{\mathcal{U}} \setminus \emptyset$. Subsets in $\mathbb{A}$ are authorized, while those not in $\mathbb{A}$ are unauthorized. An access structure is monotone if for any $Y, Z \subseteq \mathcal{U}$, $Y \in \mathbb{A}$ and $Y \subseteq Z$ imply $Z \in \mathbb{A}$ [49].

3.3. Linear secret sharing scheme (LSSS)

An LSSS scheme Π is defined by an $\ell \times n$ share-generating matrix $\mathbb{M} \in \mathbb{Z}_p^{\ell \times n}$ together with an access policy $(\mathbb{M}, \rho)$, where ρ is a function mapping each row index i of $\mathbb{M}$ to an attribute. Let $\vec{v} = (s, v_2, \dots, v_n)^T \in \mathbb{Z}_p^n$, where $s \in \mathbb{Z}_p$ is the secret and $v_2, \dots, v_n \in \mathbb{Z}_p$ are randomly chosen values. The share vector $\vec{\lambda} = \mathbb{M}\vec{v}$ defines ℓ shares of s . A user holding an attribute set S can reconstruct the secret s if and only if there exist constants $\{\omega_i \in \mathbb{Z}_p\}_{i \in I}$, where $I = \{i : \rho(i) \in S\}$, such that $\sum_{i \in I} \omega_i \lambda_i = s$ [49].

3.4. Bilinear pairing

Let $\mathbb{G}$ and $\mathbb{G}_T$ be multiplicative cyclic groups of prime order p , and let g be a generator of $\mathbb{G}$. A bilinear map $\hat{e} : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ satisfies the following properties [50]:

- **Bilinearity:** for all $g_1, g_2 \in \mathbb{G}$ and $\rho_1, \rho_2 \in \mathbb{Z}_p^*$, $\hat{e}(g_1^{\rho_1}, g_2^{\rho_2}) = \hat{e}(g_1, g_2)^{\rho_1 \rho_2}$.
- **Non-degeneracy:** there exist $g_1, g_2 \in \mathbb{G}$ such that $\hat{e}(g_1, g_2) \neq 1$.
- **Computability:** for all $g_1, g_2 \in \mathbb{G}$, $\hat{e}(g_1, g_2)$ can be efficiently computed in polynomial time.

3.5. Decisional bilinear Diffie-Hellman (DBDH) assumption

Let $(\mathbb{G}, \mathbb{G}_T, \hat{e}, g)$ be bilinear map parameters, and $a_1, a_2, a_3 \xrightarrow{\$} \mathbb{Z}_p$ be random elements of $\mathbb{Z}_p$. We say that the DBDH assumption holds if no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can distinguish $(g^{a_1}, g^{a_2}, g^{a_3}, \hat{e}(g, g)^{a_1 a_2 a_3})$ from $(g^{a_1}, g^{a_2}, g^{a_3}, Q)$ with non-negligible advantage [4], where Q is randomly selected from $\mathbb{G}_T$, i.e.,

$$Adv_{\mathcal{A}}^{\text{DBDH}} = \left| \Pr [\mathcal{A}(g^{a_1}, g^{a_2}, g^{a_3}, \hat{e}(g, g)^{a_1 a_2 a_3}) = 1] - \Pr [\mathcal{A}(g^{a_1}, g^{a_2}, g^{a_3}, Q) = 1] \right|$$

Definition 1. The DBDH assumption holds if no PPT adversary $\mathcal{A}$ can solve the DBDH problem with non-negligible advantage.

3.6. Schnorr non-interactive zero-knowledge (SNIZK)

The SNIZK proof protocol [51] allows a prover to demonstrate knowledge of a secret value to a verifier without revealing the secret itself. Let $(\mathbb{G}, g, p, H)$ be the public parameters, where H is a cryptographic hash function. The prover holds a secret $\sigma \xleftarrow{\$} \mathbb{Z}_p^*$ and publishes a public key $y = g^\sigma$.

- **Prover:** Choose $c \xleftarrow{\$} \mathbb{Z}_p^*$, compute $\mathcal{R} = g^c$, $h = H(\mathcal{R}, y)$, and $\pi = c + \sigma h$. Send the SNIZK proof $(\mathcal{R}, \pi)$ to the verifier.
- **Verifier:** Compute $h = H(\mathcal{R}, y)$ and accept if $\mathcal{R} = g^\pi y^{-h}$.
- **Correctness:** $g^\pi y^{-h} = g^{c + \sigma H(\mathcal{R}, y)} g^{-\sigma H(\mathcal{R}, y)} = g^c = \mathcal{R}$.
- **Security:** In the random oracle model, the prover must commit to $\mathcal{R}$ before learning h , preventing adaptive bias of the challenge and thus ruling out forgery.

4. Problem formulations

In this section, we present the system model, formal definition, notation, threat model, security model, and design goals.

4.1. System model

The proposed system involves the following eight entities.

- **Key Generation Center (KGC):** The KGC initializes the system by generating the global parameters (GP) and coordinating blockchain validator nodes to execute a distributed key generation (DKG) protocol that jointly derives the system master secret key (MSK). In addition, the KGC issues public-private key pairs to users and maintains version-controlled files.
- **Attribute Authorities (AAs):** Each AA manages a subset of attributes, generates the corresponding key pairs, and derives attribute secret keys SK_{uid} based on the user's attribute set S_{uid} .
- **Data Owners (DOs):** DOs define access policies, symmetrically encrypt data files, encrypt access tokens using ABE in a hybrid on-chain/off-chain manner, and construct searchable encrypted keyword indexes. The encrypted files are outsourced to the cloud storage provider (CSP), while the encrypted access tokens and keyword indexes are stored in the blockchain ledger.
- **Data Users (DUs):** DUs generate trapdoors corresponding to their keywords of interest and submit them to the blockchain. Upon receiving the search results and an aggregated SNIZK proof from the CSP, the DU invokes a smart contract to verify the proof. The DU then decrypts the partially encrypted access token from the edge server (ES), derives the file encryption key, and finally performs symmetric decryption to recover the target files.
- **Cloud Storage Provider (CSP):** Leveraging its computational capability, the CSP executes smart contracts to retrieve multi-keyword search results using the encrypted indexes and trapdoors stored on-chain, and returns an aggregated SNIZK proof for verification.
- **Edge Server (ES):** The ES checks whether the DU's attributes satisfy the DO's access policy. If so, it partially decrypts the access token's ciphertext and returns the results to the DU for final decryption.
- **Blockchain Platform (BCT):** The BCT functions as a tamper-resistant distributed ledger [52] that immutably recording file hash addresses, encrypted keyword indexes, and trapdoors, and deploys smart contracts.
- **Revocation and Tracing Authority (RTA):** The RTA maintains a user registration list (RegList) and a user tracing list (TraceList), which support user revocation and the tracing of misbehaving users whose secret keys are leaked or misused.

The workflow of the proposed BAMKS system is illustrated in Fig. 1 and proceeds as follows. First, the KGC publishes the global parameters GP and collaborates with blockchain validator nodes to derive the system master secret key MSK using DKG (step ①). Each AA then employs GP to generate its public-private key pairs for the attributes under its control (step ②). Subsequently, the KGC distributes the multi-owner group master key to the DOs (step ③) and issues public-private key pairs to the DUs (step ④). Meanwhile, the RTA maintains two state lists: a user registration list (RegList) that records all legitimate DUs and assigns them a shared membership control factor, and a user tracing list (TraceList) that later stores trace credentials generated during secret key

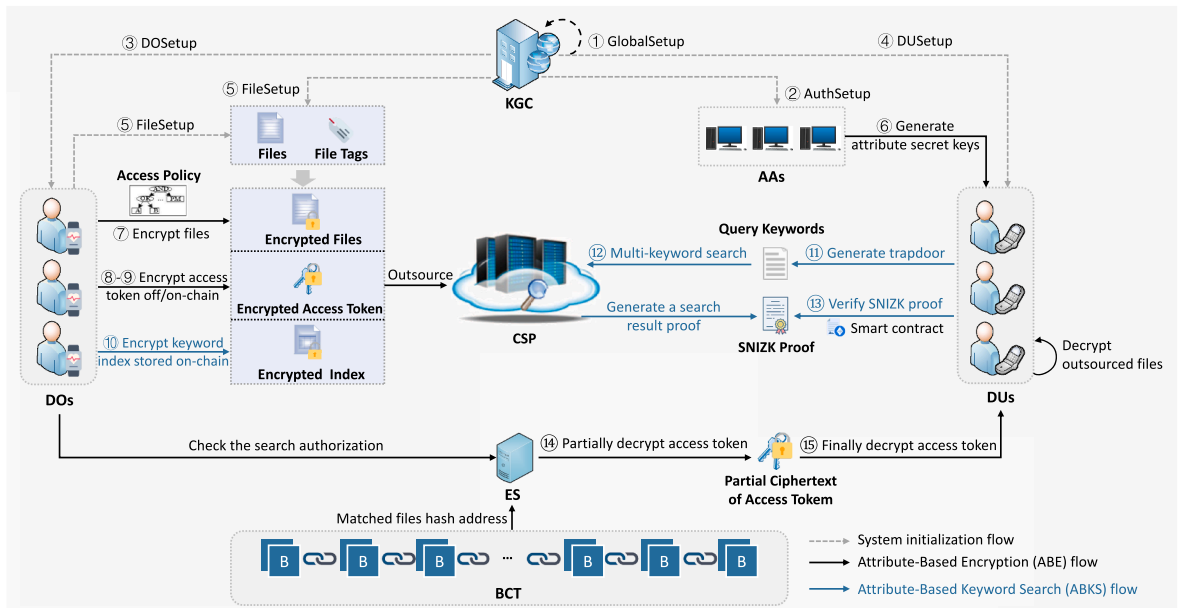


Fig. 1. System model.

Table 1

Notations and descriptions.

Notation	Description	Notation	Description
ι	Security parameter.	$\mathcal{F}_{ver}$	File set with a specific version $\{f_{1,ver}, f_{2,ver}, \dots, f_{L,ver}\}$.
GP	Global public parameters.	VMK_{ver}	Version master keys for the file set, consisting of $\{\delta_{ver}, dsk_{ver}\}$.
MSK	Master secret key of the system.	VPK_{ver}	Version public keys of the file set, consisting of $\{g^{\delta_{ver}}, dpk_{ver}\}$.
$H(\cdot)$	Hash function mapping to G , $H : \{0, 1\}^* \rightarrow G$.	δ_{ver}	Version-specific master key of the file set $\mathcal{F}_{ver}$.
$H_1(\cdot)$	Hash function mapping to $\mathbb{Z}_p^*$, $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$.	(dpk_{ver}, dsk_{ver})	Public/secret key pair for the versioned file set $\mathcal{F}_{ver}$.
$\mathcal{U}, \mathcal{U}_\theta$	Attribute/authority universe.	$(\sigma_\kappa, y_\kappa)$	Public/secret tag pair of document κ in a file set.
$(pk_{AA_\theta}, sk_{AA_\theta})$	Public/secret key pair of authority AA_θ .	$(\mathbb{M}, \rho)$	Access structure with matrix $\mathbb{M}$ and mapping function ρ .
(upk, usk)	Public/secret key pair of a user.	S	Attribute set used in an access policy.
$\mathcal{O}$	Set of data owners $\{O_i\}_{i=1}^N$.	S_{uid}	Attribute set of user uid , $\{att_1, att_2, \dots, att_x\}$.
h_m	Membership control factor.	$\mathcal{I}$	Attribute set used for decryption, $\mathcal{I} \subseteq S$.
ϵ	Multi-owner group master key.	$CT_{\mathcal{F}_{ver}}$	Ciphertext of file set $\mathcal{F}_{ver}$.
Φ	Access token.	ICT_Φ/CT_Φ	Intermediate/complete ciphertext of access token Φ .
$\mathcal{W}$	Keyword dictionary $\{w_1, w_2, \dots, w_m\}$.	(tpk, ts_k)	Outsourced decryption public/secret key pair.
$\tilde{\mathcal{W}}$	Query keyword set $\{\tilde{w}_1, \tilde{w}_2, \dots, \tilde{w}_m\}$.	PCT_Φ	Partially decrypted ciphertext of access token Φ .
Index$_{\mathcal{W}}$	Encrypted keyword index of $\mathcal{W}$.	RegList, TraceList	User registration/tracing list.
Trap$_{\tilde{\mathcal{W}}}$	Trapdoor for query keyword set $\tilde{\mathcal{W}}$.	$\hat{\pi}$	Aggregated SNIZK proof.

issuance. Afterwards, the KGC, together with the DOs, initializes the version-controlled file set (step ⑤). Within the ABE framework, AAs generate attribute secret keys based on the user's attributes (step ⑥). DOs then symmetrically encrypt the data files (step ⑦) and protect the corresponding access tokens via a hybrid on-chain/off-chain ABE encryption process (steps ⑧–⑨). Additionally, keywords are encrypted to generate searchable indexes (step ⑩). The encrypted files are outsourced to the CSP, while the encrypted access tokens and keyword indexes are stored on the BCT. For data retrieval, a DU generates a trapdoor for the interested keywords (step ⑪) and submits it to the ES. The ES first verifies the DU's authorization against the DO's predefined access policy. Upon successful verification, the ES forwards the trapdoor to the CSP, which retrieves the encrypted indexes from the BCT and performs the search computation by matching the user's trapdoor against them (step ⑫). The search results are accompanied by an aggregated SNIZK proof, which the CSP sends to the DU. The DU can then verify the completeness of the results by executing the *Result Verification Smart Contract (RVSC)* (step ⑬). Once verification succeeds, the ES partially decrypts the access token's ciphertext (step ⑭) and sends the partially decrypted result to the DU. The DU then completes the final decryption of the access token (step ⑮), derives the corresponding symmetric key, and recovers the plaintext files. In addition to index updating for version-controlled files, BAMKS also supports attribute revocation, user revocation, and user traceability during dynamic updates. These dynamic control capabilities are realized through the *KeyUpdate*($\cdot$), *CTUpdate*($\cdot$), and *Trace*($\cdot$) algorithms (see Sections 5.8 and 5.9), and are triggered only when needed; therefore, they are not explicitly shown in Fig. 1 for simplicity.

4.2. Notation

Table 1 presents the main notations along with their descriptions.

4.3. Formal definition

The proposed BAMKS scheme consists of the following algorithms:

- (1) *GlobalSetup*(1^ι) $\rightarrow (GP, MSK)$: On input of a security parameter ι , the KGC runs *GlobalSetup*($\cdot$) to generate the global public parameters GP and coordinates a committee of blockchain validator nodes to jointly derive the system master secret key MSK via a distributed key generation (DKG) protocol. The GP are published on-chain, while the MSK shares are kept secret by the validator nodes.
- (2) *AuthSetup*($GP, \mathcal{U}_\theta, \theta$) $\rightarrow (pk_{AA_\theta}, sk_{AA_\theta})$: Given GP and the attribute universe $\mathcal{U}_\theta$, each attribute authority AA_θ executes *AuthSetup*($\cdot$) to generate a public-private key pair $(pk_{AA_\theta}, sk_{AA_\theta})$ for the attributes under its control.
- (3) *DOSetup*($GP, \mathcal{O}$) $\rightarrow (\epsilon, \Phi)$: Given GP and the set of data owners $\mathcal{O}$, the KGC executes *DOSetup*($\cdot$) to generate a multi-owner group master key ϵ and the access token Φ .
- (4) *DUSetup*(GP, uid) $\rightarrow ((upk, usk), h_m, \text{RegList})$: Given GP and a user identifier uid , the KGC runs *DUSetup*($\cdot$) to generate the user's public-private key pair (upk, usk) . Meanwhile, the RTA issues a membership control factor h_m shared by all non-revoked users recorded in the user registration list *RegList*.
- (5) *FileSetup*($GP, \mathcal{F}_{ver}, \epsilon$) $\rightarrow ((VPK_{ver}, VMK_{ver}), \{(\sigma_\kappa, y_\kappa)\})$: Given GP , a versioned file set $\mathcal{F}_{ver}$, and the multi-owner group key ϵ , the DOs run *FileSetup*($\cdot$) to create the version public/secret key pair (VPK_{ver}, VMK_{ver}) and a public/secret tag pair $(\sigma_\kappa, y_\kappa)$ for each file.
- (6) *KeyGen*($GP, uid, S_{uid}, \{sk_{AA_\theta}\}, dpk_{ver}, h_m$) $\rightarrow SK_{uid}$: Given GP , a user identifier uid , the user's attribute set S_{uid} , the corresponding private keys of relevant AAs $\{sk_{AA_\theta}\}$, the public key of the file set dpk_{ver} , and the shared membership control factor h_m , the AAs collaboratively execute *KeyGen*($\cdot$) to derive the user's attribute secret key SK_{uid} .
- (7) *FileEnc*($\mathcal{F}_{ver}, \mathcal{K}, \Phi, VPK_{ver}$) $\rightarrow CT_{\mathcal{F}_{ver}}$: Given $\mathcal{F}_{ver}$, the symmetric encryption key $\mathcal{K}$, the access token Φ , and VPK_{ver} , the DOs execute *FileEnc*($\cdot$) to obtain the ciphertext set $CT_{\mathcal{F}_{ver}}$.
- (8) *TokenEnc_{off-chain}*($GP, \{pk_{AA_\theta}\}, dsk_{ver}, h_m$) $\rightarrow ICT_\Phi$: Given GP , $\{pk_{AA_\theta}\}$, dsk_{ver} , and h_m , the DOs execute *TokenEnc_{off-chain}*($\cdot$) to produce the intermediate ciphertext ICT_Φ of the access token Φ .

- (9) $\text{TokenEnc}_{\text{on-chain}}(GP, (\mathbb{M}, \rho), \Phi, ICT_\Phi) \rightarrow CT_\Phi$: Given GP , an access structure $(\mathbb{M}, \rho)$, the token Φ , and the intermediate ciphertext ICT_Φ , the DOs invoke $\text{TokenEnc}_{\text{on-chain}}(\cdot)$ to obtain the final ciphertext CT_Φ .
- (10) $\text{IndexGen}(GP, \mathcal{W}, \mathcal{O}) \rightarrow \text{Index}_\mathcal{W}$: Given GP , the keyword set $\mathcal{W}$ extracted from F_{ver} , and the data owner group $\mathcal{O}$, the DOs execute $\text{IndexGen}(\cdot)$ to generate the encrypted keyword index $\text{Index}_\mathcal{W}$, which is then stored on the BCT.
- (11) $\text{TrapGen}(GP, \mathcal{W}, usk) \rightarrow \text{Trap}_\mathcal{W}$: Given GP , a query keyword set $\mathcal{W}$, and the user's private key usk , the DU executes $\text{TrapGen}(\cdot)$ to generate a search trapdoor $\text{Trap}_\mathcal{W}$.
- (12) $\text{Search}(GP, C_0, \text{Index}_\mathcal{W}, \text{Trap}_\mathcal{W}) \rightarrow \{\text{SRL}, \perp\}$: Given GP , $C_0 \in CT_\Phi$, $\text{Index}_\mathcal{W}$, and $\text{Trap}_\mathcal{W}$, the CSP executes $\text{Search}(\cdot)$ to check for the matching entries, outputting a search result list SRL consisting of the indices of the matched ciphertexts, or $\perp$ if no match is found.
- (13) $\text{ProofVerify}(GP, \text{SRL}, \{y_{k^*}\}) \rightarrow \{\checkmark / \perp\}$: Given GP , SRL, and the corresponding file tags $\{y_{k^*}\}$, the CSP first generates an aggregated SNIZK proof $\hat{\pi}$. The DU subsequently invokes the *Result Verification Smart Contract (RVSC)* to verify $\hat{\pi}$ by executing $\text{ProofVerify}(\cdot)$, which outputs $\checkmark$ if the search results are correct and complete, and $\perp$ otherwise.
- (14) $\text{PartialDecrypt}(GP, CT_\Phi, SK_{uid}) \rightarrow PCT_\Phi$: Given GP , the DU locally generates an outsourced decryption key pair (tpk, tsk) . Then, using CT_Φ , SK_{uid} , and tpk , the ES runs $\text{PartialDecrypt}(\cdot)$ to produce the partially decrypted access token PCT_Φ .
- (15) $\text{FinalDecrypt}(GP, PCT_\Phi, VPK_{\text{ver}}, tsk, CT_{F_{\text{ver}}^*}) \rightarrow F_{\text{ver}}^*$: Given GP , PCT_Φ , VPK_{ver} , tsk , and the ciphertexts of the matched files $CT_{F_{\text{ver}}^*}$, the DU executes $\text{FinalDecrypt}(\cdot)$ to recover Φ , derive the symmetric key K_{ver} , and finally decrypt the matched file set F_{ver}^* .
- (16) $\text{KeyUpdate}(GP, SK_{uid}, \Gamma_{\text{upd}}) \rightarrow \overline{SK}_{uid}$: Given GP , SK_{uid} , and an update factor Γ_{upd} , the authority computes the refreshed secret key $\overline{SK}_{uid}$. (The specific form of Γ_{upd} depends on whether it corresponds to attribute or user revocation; see Section 5.8.)
- (17) $\text{CTUpdate}(GP, CT_\Phi, \Gamma_{\text{upd}}) \rightarrow \overline{CT}_\Phi$: Given GP , CT_Φ , and the corresponding Γ_{upd} , the DO refreshes the ciphertext to $\overline{CT}_\Phi$, ensuring that revoked attributes or users cannot decrypt updated ciphertexts.
- (18) $\text{KeySanityCheck}(\overline{SK}_{uid}) \rightarrow \{0, 1\}$: Given a suspected leaked key $\overline{SK}_{uid}$, the RTA verifies whether the key is structurally valid.
- (19) $\text{Trace}(\overline{SK}_{uid}) \rightarrow uid$: If $\text{KeySanityCheck}(\overline{SK}_{uid}) = 1$, the RTA identifies the misbehaving user by matching the corresponding trace credential in the tracing list TraceList and triggers user revocation.
- (20) $\text{IndexUpdate}(GP, \text{Index}_\mathcal{W}) \rightarrow \text{Index}'_\mathcal{W}$: Given GP and $\text{Index}_\mathcal{W}$, the DO executes $\text{IndexUpdate}(\cdot)$ to update the encrypted index, producing $\text{Index}'_\mathcal{W}$ using the latest file version master keys δ'_{ver} and ξ'_{ver} .

4.4. Threat model

Our system is based on the following threat model, which specifies the trust assumptions for each entity as follows.

- KGC, DOs, BCT, and RTA are assumed to be fully trusted. They are expected to honestly and accurately adhere to all protocols throughout the entire system.
- Rather than assuming a purely semi-trusted CSP, we model it as a rational adversary. It is semi-honest with respect to data privacy, namely that it stores encrypted files correctly while attempting to infer information from the observed data. Nonetheless, it may act maliciously regarding result integrity. Motivated by saving computation resources, the CSP may execute only a fraction of the search operations or even fabricate results, such as returning an incomplete set of files that match the user's query.
- AAs, ES, and DUs are honest-but-curious. Specifically, AA honestly issues attribute secret keys to users, but is curious about the data being accessed using the attributes they manage. ES honestly executes the partial decryption protocol on access tokens, but is curious about their underlying content. DUs genuinely generate and submit trapdoors for their queries, but may attempt to glean additional information from the search results beyond their authorized access privileges.

4.5. Security model

The BAMKS is required to resist the *chosen-plaintext attacks (CPA)* and *chosen keyword attacks (CKA)*.

4.5.1. IND-CPA

The CPA security game, played between a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, is defined as follows. The goal is to achieve IND-CPA security for CT_Φ under a target access structure $\mathbb{A}^*$.

- **Initialization:** $\mathcal{A}$ may corrupt a subset of AAs and obtain their secret keys $\{sk_{AA_\theta}\}$, while the remaining AAs remain honest under $\mathcal{C}$'s control. $\mathcal{A}$ then submits a challenge access structure $\mathbb{A}^* = (\mathbb{M}_{\mathcal{C}^* \times \mathcal{N}^*}, \rho^*)$ to $\mathcal{C}$.
- **Setup:** $\mathcal{C}$ runs $\text{GlobalSetup}(1')$ to generate GP and MSK . The public parameters GP are given to $\mathcal{A}$, while MSK is kept secret.
- **Query Phase 1:** $\mathcal{A}$ adaptively submits a sequence of user attribute sets $\{S_{uid}\}$ subject to the restriction that $S_{uid} \not\models \mathbb{A}^*$. For each query, $\mathcal{C}$ responds by executing $\text{KeyGen}(\cdot)$ and returns the corresponding secret key SK_{uid} to $\mathcal{A}$.
- **Challenge:** $\mathcal{A}$ selects two equal-length messages m_0 and m_1 . $\mathcal{C}$ chooses a random bit $\tau \in \{0, 1\}$ and runs the $\text{Encrypt}(\cdot)$ algorithm (covering both $\text{TokenEnc}_{\text{off-chain}}$ and $\text{TokenEnc}_{\text{on-chain}}$) under $\mathbb{A}^*$ to produce the challenge ciphertext CT^* , which is returned to $\mathcal{A}$.
- **Query Phase 2:** $\mathcal{A}$ continues to query additional attribute secret keys for sets S_{uid} that do not satisfy $\mathbb{A}^*$, as in **Query Phase 1**.
- **Guess:** $\mathcal{A}$ gives a guess $\tau' \in \{0, 1\}$ and wins the game if and only if $\tau' = \tau$. The advantage of $\mathcal{A}$ in this security game is defined as $\text{Adv}_{\mathcal{A}}^{\text{CPA}} = \left| \Pr[\tau' = \tau] - \frac{1}{2} \right|$.

Definition 2. The BAMKS scheme achieves IND-CPA security if no PPT adversary $\mathcal{A}$ can win the above game with non-negligible advantage ϵ under the DBDH assumption.

4.5.2. IND-CKA

The CKA security game, played between a PPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, is defined as follows. The goal is to achieve IND-CKA security for the keyword search functionality.

- **Setup:** $\mathcal{C}$ runs $\text{GlobalSetup}(1')$ to generate GP and MSK . The public parameters GP is given to $\mathcal{A}$, while MSK is kept secret.
- **Query Phase 1:** $\mathcal{A}$ adaptively submits a series of query keyword sets $\{\tilde{\mathcal{W}}\}$ of its choice. For each query, $\mathcal{C}$ executes $\text{TrapGen}(\cdot)$ and returns the corresponding trapdoor $\text{Trap}_{\tilde{\mathcal{W}}}$ to $\mathcal{A}$.
- **Challenge:** $\mathcal{A}$ submits two challenge keyword dictionaries $\mathcal{W}_0$ and $\mathcal{W}_1$, both of equal length and containing the same number of keywords, neither of which were used in **Query Phase 1**. $\mathcal{C}$ picks a random bit $\tau \in \{0, 1\}$ and runs $\text{IndexGen}(\cdot)$ to generate the encrypted index $\text{Index}_{\mathcal{W}_\tau}$, which is then returned to $\mathcal{A}$.
- **Query Phase 2:** $\mathcal{A}$ may continue to issue queries for additional keyword sets, as in **Query Phase 1**, with the restriction that $\mathcal{W}_0$ and $\mathcal{W}_1$ cannot be queried again.
- **Guess:** $\mathcal{A}$ outputs a guess $\tau' \in \{0, 1\}$ and wins the game if and only if $\tau' = \tau$. Its advantage is defined as $\text{Adv}_{\mathcal{A}}^{\text{CKA}} = \left| \Pr[\tau' = \tau] - \frac{1}{2} \right|$.

Definition 3. The BAMKS achieves IND-CKA security if no PPT adversary $\mathcal{A}$ can win the above game with non-negligible advantage ϵ under the DBDH assumption.

4.6. Design goals

BAMKS enables secure and flexible encrypted data access in cloud-edge-IoT environments with the following objectives:

- **Dynamic and Fine-Grained Access Control:** Enable multiple data owners to securely share dynamically evolving data while ensuring that only users whose attributes satisfy the specified access policy can obtain decryption privileges. Access rights can be updated over time without reissuing keys to all users.
- **Efficient Conjunctive Multi-Keyword Search:** Support expressive and accurate keyword-based retrieval over encrypted files via conjunctive search, while ensuring that neither keyword queries nor index contents are revealed to the server.
- **Privacy-Preserving Result Verification:** Allow users to verify the correctness and completeness of returned search results by employing zero-knowledge proofs, without disclosing plaintext content or search queries.
- **Lightweight Revocation and User Traceability:** Support attribute and user revocation by updating only small key and ciphertext components, without reconstructing secret keys or re-encrypting existing files, thereby achieving forward security at minimal cost. Moreover, identify malicious users associated with leaked decryption keys, enabling accountable access control.

5. Detailed construction

5.1. System initialization

(1) $\text{GlobalSetup}(1') \rightarrow (GP, MSK)$

Let $\mathbb{G}$ and $\mathbb{G}_T$ be multiplicative cyclic groups of prime order p , and let $\hat{e} : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ be a bilinear map with generator $g \in \mathbb{G}$. The bilinear map parameters are $pp = (p, g, \hat{e}, \mathbb{G}, \mathbb{G}_T, \hat{e}(g, g))$. Define two cryptographically secure hash functions $H : \{0, 1\}^* \rightarrow \mathbb{G}$ and $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$. To eliminate any single-point-of-failure, the system adopts a distributed key generation (DKG) protocol executed by a set of blockchain validator nodes $\mathcal{N}$ under the coordination of the KGC. Each node $n \in \mathcal{N}$ independently samples $\mu_n, \gamma_n \xleftarrow{\$} \mathbb{Z}_p^*$ and publishes the commitments $\text{Com}_{\mu, n} = g^{\mu_n}$ and $\text{Com}_{\gamma, n} = g^{\gamma_n}$ on-chain. Based on these commitments, the accumulated public values are defined as $g^\mu = \prod_{n \in \mathcal{N}} \text{Com}_{\mu, n}$, $g^\gamma = \prod_{n \in \mathcal{N}} \text{Com}_{\gamma, n}$, $\hat{e}(g^\mu, g) = \hat{e}(g, g)^\mu$. The system master secret key is implicitly computed as $\mu = \sum_{n \in \mathcal{N}} \mu_n \bmod p$, $\gamma = \sum_{n \in \mathcal{N}} \gamma_n \bmod p$, so that no single node can reconstruct it. The smart contract stores $(g^\mu, g^\gamma, \hat{e}(g, g)^\mu)$ on-chain, enabling auditability of the system initialization process. Meanwhile, each validator node holds only its own secret key shares (μ_n, γ_n) , ensuring that no party learns the entire MSK . Consequently,

$$\begin{aligned} GP &= \{pp, H, H_1, \hat{e}(g, g)^\mu, g^\mu, g^\gamma\}, \\ MSK &= \{\mu, \gamma\}. \end{aligned} \quad (1)$$

Remark 1. Unlike centralized schemes that rely on a fully trusted KGC to generate the system master secret key MSK , BAMKS adopts a DKG protocol executed by blockchain validator nodes. As a result, MSK (i.e., μ and γ) cannot be reconstructed by any individual node, since each validator only retains its own key share. This decentralized initialization avoids a single-point-of-failure and prevents any individual party from compromising the system.

(2) $\text{AuthSetup}(GP, \mathcal{U}_\Theta, \theta) \rightarrow (pk_{AA_\theta}, sk_{AA_\theta})$

For the set of attributes managed by an authority $AA_\theta \in \mathcal{U}_\Theta$, the authority samples random values $\alpha_\theta, \beta_\theta \xleftarrow{\$} \mathbb{Z}_p^*$. It sets the secret key as $sk_{AA_\theta} = \{\alpha_\theta, \beta_\theta\}$ and computes the corresponding public key as $pk_{AA_\theta} = \{\hat{e}(g, g)^{\alpha_\theta}, g^{\beta_\theta}\}$.

(3) $\text{DOSetup}(GP, \mathcal{O}) \rightarrow (\epsilon, \Phi)$

Given the set of DOs $\mathcal{O} = \{O_1, O_2, \dots, O_N\}$, the KGC executes $\text{DOSetup}(\cdot)$ to choose a multi-owner group master key $\epsilon \xleftarrow{\$} \mathbb{Z}_p^*$. For each $O_r \in \mathcal{O}$, the KGC further selects a random $\varphi_r \xleftarrow{\$} \mathbb{Z}_p^*$, and computes the access token as $\Phi = \sum_{r=1}^N \varphi_r$.

(4) $\text{DUSetup}(GP, uid) \rightarrow ((upk, usk), h_w, \text{RegList}, \text{TraceList})$

For each DU with identifier uid , the KGC selects two randoms $\chi, z \xleftarrow{\$} \mathbb{Z}_p^*$ and derives the user's public-private key pair as $upk = \{g^\chi, g^z\}$ and $usk = \{\chi, z\}$. The KGC then securely delivers usk to the corresponding user. Meanwhile, the RTA maintains a user registration list (RegList) that records currently legitimate users and assigns a global membership control factor $h_w = H_1(w)$, where $w \leftarrow \mathbb{Z}_p^*$. This factor is shared among all non-revoked users and is refreshed whenever the membership of RegList changes. The RTA also maintains a user tracing list (TraceList), which will later store trace credentials $\mathcal{T}_{uid}$ generated during the **Key Generation** phase (see Section 5.2). Finally, the RTA appends (uid, upk, h_w) to RegList and initializes the corresponding entry in TraceList for later recording of trace credentials.

(5) $\text{FileSetup}(GP, F_{ver}, \epsilon) \rightarrow ((VPK_{ver}, VMK_{ver}), \{(\sigma_k, y_k)\})$

Given a versioned file set $F_{ver} = \{f_{1,ver}, \dots, f_{L,ver}\}$ contributed by multiple DOs, the KGC executes FileSetup($\cdot$) to derive parameters:

- For each version $ver \in [1, V]$ of the file set, it derives the version master key $VMK_{ver} = \{\delta_{ver}, dsk_{ver}\}$ and the version public key $VPK_{ver} = \{g^{\delta_{ver}}, dpk_{ver}\}$, where $\delta_{ver} \xleftarrow{\$} \mathbb{Z}_p^*$ is the version-specific master key, $g^{\delta_{ver}}$ serves as the file set identifier, $dsk_{ver} = \delta_{ver}/\epsilon$ is the file set secret key, and $dpk_{ver} = g^{\delta_{ver}/\epsilon}$ is the corresponding public key.
- For each document $f_{k,ver} \in F_{ver}$, the associated DO randomly chooses $\sigma_k \xleftarrow{\$} \mathbb{Z}_p^*$ and computes $y_k = g^{\sigma_k}$. The resulting public-secret tag pair (σ_k, y_k) will be used to verify the correctness of search results during the **Result Verification** phase (see Section 5.6).

5.2. Key generation

(6) $\text{KeyGen}(GP, uid, S_{uid}, \{sk_{AA_\theta}\}, dpk_{ver}, h_w) \rightarrow SK_{uid}$

The algorithm picks a random $t \xleftarrow{\$} \mathbb{Z}_p^*$. For each attribute $att_i \in S_{uid}$ managed by authority AA_θ with secret key $sk_{AA_\theta} = (\alpha_\theta, \beta_\theta)$, the following user attribute secret keys are computed:

$$\begin{aligned} K_1 &= H(uid)^{-t} \\ K_{2,i} &= H(uid)^{t\beta_i} dpk_{ver}^{\alpha_i} = H(uid)^{t\beta_i} g^{\frac{\delta_{ver}\alpha_i}{\epsilon}} \\ K_3 &= h_w \\ SK_{uid} &= (K_1, \{K_{2,i}\}_{i \in S_{uid}}, K_3). \end{aligned} \quad (2)$$

Remark 2. (i) **Revocation.** h_w is a membership control factor recorded in the user registration list (RegList). It is used during outsourced decryption to confirm that a requesting user remains valid (i.e., non-revoked). When a user is removed from RegList, h_w is refreshed and serves as the update factor for efficiently rekeying user secret keys and refreshing ciphertexts.

(ii) **Traceability.** To enable accountability, the RTA computes a tracing credential for each user, $\mathcal{T}_{uid} = \hat{e}(K_1^{-1}, g) = \hat{e}(H(uid), g)^t \in \mathbb{G}_T$, and records it in the tracing list (TraceList).

5.3. Ciphertexts uploading

(7) $\text{FileEnc}(F_{ver}, \mathcal{K}, \Phi, VPK_{ver}) \rightarrow CT_{F_{ver}}$

Each DO employs a symmetric key $\mathcal{K}$ (e.g., AES) to encrypt the file set $F_{ver} = \{f_{k,ver} \mid k \in [1, L]\}$. To support both dynamic ownership and version control, the DOs derive a re-encryption key:

$$\mathcal{K}'_{ver} = \text{KDF}(\Phi \parallel H_1(g^{\delta_{ver}})), \quad (3)$$

where $\Phi = \sum_{r=1}^N \varphi_r$ (hereinafter referred to as the *access token*) is generated during the DOSetup phase, and $g^{\delta_{ver}} \in VPK_{ver}$ is the version-specific identifier. The derived key $\mathcal{K}'_{ver}$ is used to wrap the file encryption key $\mathcal{K}$, thus yielding the ciphertext:

$$CT_{F_{ver}} = \text{FileEnc}(F_{ver}, \mathcal{K}, \Phi, VPK_{ver}). \quad (4)$$

Remark 3. The file set F_{ver} is encrypted only once under $\mathcal{K}$, while access to $\mathcal{K}$ is mediated by the access token Φ and the version identifier $g^{\delta_{ver}}$. This design provides two layers of protection: (i) when the set of data owners changes, only Φ needs to be updated, without re-encrypting stored files; (ii) when a new file version is released, the derived key $\mathcal{K}'_{ver}$ changes due to $g^{\delta_{ver}}$, ensuring that outdated keys cannot decrypt newer versions. Thus, the scheme naturally supports dynamic ownership and fine-grained version control with minimal rekeying overhead.

(8) $\text{TokenEnc}_{off-chain}(GP, \{pk_{AA_\theta}\}, dsk_{ver}, h_w) \rightarrow ICT_\Phi$

The access token Φ is protected via a hybrid on-chain/off-chain ABE encryption mechanism. Before defining the access policy, DOs perform the off-chain encryption step. Given $\{pk_{AA_\theta}\}$, dsk_{ver} , and h_w , the algorithm selects a random value $r_x \xleftarrow{\$} \mathbb{Z}_p^*$ for each row x of the access matrix $\mathbb{M}$ and computes the intermediate ciphertext of Φ as follows.

$$\begin{aligned} C_{1,x} &= g^{\beta_{\rho(x)} r_x}, \quad C_{2,x} = g^{r_x}, \\ C_{3,x} &= \hat{e}(g, g)^{\frac{a_{\rho(x)} r_x \cdot dsk_{ver}}{h_w}} = \hat{e}(g, g)^{\frac{a_{\rho(x)} r_x \delta_{ver}}{\epsilon h_w}}, \\ ICT_\Phi &= \{C_{1,x}, C_{2,x}, C_{3,x}\}_{x \in [1, \ell]}. \end{aligned} \quad (5)$$

(9) $\text{TokenEnc}_{\text{on-chain}}(GP, (\mathbb{M}, \rho), \Phi, ICT_\Phi)$

The DOs define an LSSS access policy $(\mathbb{M}, \rho)$ [49], where $\mathbb{M}$ is an $\ell \times n$ matrix and $\rho(\cdot)$ maps each row $\mathbb{M}_x$ to its associated attribute [53]. The access token Φ is encrypted on-chain as follows: select a random $s \leftarrow \mathbb{Z}_p^*$ and $n-1$ random values $v_2, \dots, v_n \in \mathbb{Z}_p^*$ to form $\vec{v} = (s, v_2, \dots, v_n)^\top$, and compute $\lambda_x = \mathbb{M}_x \cdot \vec{v}$ for each row $\mathbb{M}_x$ of $\mathbb{M}$.

Finally, compute C , C_0 and $C_{4,x}$ to obtain the final ciphertext CT_Φ :

$$C = \Phi \cdot \hat{e}(g, g)^{\mu s}, \quad C_0 = g^s, \quad C_{4,x} = g^{\mu \lambda_x}$$

$$CT_\Phi = (C, C_0, \{C_{1,x}, C_{2,x}, C_{3,x}, C_{4,x}\}_{x \in [1, \ell]}).$$
 (6)

Remark 4. (i) Most computationally intensive encryption are performed off-chain, whereas only lightweight computations are executed on-chain. (ii) The intermediate ciphertext ICT_Φ needs to be generated only once and can be reused for multiple on-chain encryptions, substantially reducing the overall ABE cost.

(10) $\text{IndexGen}(GP, \mathcal{W}, \mathcal{O}) \rightarrow \text{Index}_{\mathcal{W}}$

Given a keyword dictionary $\mathcal{W} = \{w_1, \dots, w_m\}$ with version ver , let ξ_{ver} denote the dictionary master key. Each DO selects a random $\eta_r \xleftarrow{\$} \mathbb{Z}_p^*$, and the aggregate value $\eta = \sum_{r=1}^N \eta_r$ is computed for the multi-owner group $\mathcal{O}$. If the keyword $w_j \in \mathcal{W}$ is contained in file $f_{\kappa, ver}$, then $I_{\kappa, j} = g^{\eta H_1(w_j)}$; otherwise $I_{\kappa, j} = 1$. The keyword index is constructed as:

$$I_1 = g^{\eta + s},$$

$$I_{2,r} = g^{\eta_r},$$

$$\text{Index}_{\mathcal{W}} = (I_1, \{I_{2,r}\}_{r \in [1, N]}, \{I_{\kappa, j}\}_{\kappa \in [1, L], j \in [1, m]}).$$
 (7)

5.4. Trap generation

(11) $\text{TrapGen}(GP, \tilde{\mathcal{W}}, usk) \rightarrow \text{Trap}_{\tilde{\mathcal{W}}}$

Given a keyword query set $\tilde{\mathcal{W}} = \{\tilde{w}_1, \dots, \tilde{w}_{m^*}\} \subseteq \mathcal{W}$, the DU runs $\text{TrapGen}(\cdot)$ to generate the trapdoor $\text{Trap}_{\tilde{\mathcal{W}}}$. To achieve trapdoor unlinkability (i.e., identical queries produce different trapdoors), the DU selects a random $\phi \xleftarrow{\$} \mathbb{Z}_p^*$ and computes:

$$T_1 = g^{\gamma \phi},$$

$$T_2 = g^{\phi + \frac{\gamma}{z}},$$

$$T_3 = (g^{\gamma})^{\frac{\gamma}{\sum_{j=1}^{m^*} H_1(\tilde{w}_j)}}$$

$$\text{Trap}_{\tilde{\mathcal{W}}} = \{T_1, T_2, T_3\}.$$
 (8)

The DU then sends the resulting $\text{Trap}_{\tilde{\mathcal{W}}}$ to the ES.

5.5. Keywords retrieval

(12) $\text{Search}(GP, C_0, \text{Index}_{\mathcal{W}}, \text{Trap}_{\tilde{\mathcal{W}}}) \rightarrow \{\text{SRL}, \perp\}$

Upon receiving a query request from the DU, the nearest ES first verifies whether the user's attribute set satisfies the access policy of the requested files and checks if the DU has a valid h_m associated with the current RegList . If authorized, the ES forwards the trapdoor to the CSP. The CSP then executes the $\text{Search}(\cdot)$ algorithm to perform a conjunctive multi-keyword search, which matches the trapdoor $\text{Trap}_{\tilde{\mathcal{W}}}$ against the keyword indexes $\text{Index}_{\mathcal{W}}$, i.e., by verifying whether the following equation holds:

$$\frac{\hat{e}\left(\prod_{r=1}^N I_{2,r}, T_2\right)}{\hat{e}(I_1/C_0, T_1)} \stackrel{?}{=} \hat{e}\left(\prod_{j^*=1}^{m^*} I_{\kappa, j^*}, T_3\right)$$
 (9)

If the equation holds, the algorithm outputs a search result list SRL containing the indices of the files that match the query. Otherwise, it outputs $\perp$.

5.6. Results verification

(13) $\text{ProofVerify}(GP, \text{SRL}, \{y_{\kappa^*}\}) \rightarrow \{\checkmark / \perp\}$

Since the CSP is semi-trusted, it is essential to allow the DU to verify the integrity and correctness of the search result set $F_{ver}^* = \{f_{\kappa^*, ver} | \kappa^* \in [1, L^*]\}$, without revealing any underlying file content. To this end, we construct an aggregated Schnorr-based non-interactive zero-knowledge (SNIZK) proof [51], which enables efficient batch verification of matched files while preserving data privacy. Specifically, let $\{\kappa^*\}_{1 \leq \kappa^* \leq L^*}$ denote the set of matched file indices returned in the search result list SRL. For each κ^* in this set, the CSP retrieves the public tag $y_{\kappa^*} = g^{\sigma \kappa^*}$ associated with the corresponding file $f_{\kappa^*, ver}$. The $\text{ProofVerify}(\cdot)$ procedure is executed by the *Result Verification Smart Contract (RVSC)* (see Algorithm 1) and is described in detail below.

(a) **CSP (Prover):**

- Select a random $c_{\kappa^*} \xleftarrow{\$} \mathbb{Z}_p^*$ for each matched file.
- Compute $\mathcal{R} = g^{\sum_{\kappa^*=1}^{L^*} c_{\kappa^*}}$.
- For each file, compute $h_{\kappa^*} = H_1(g \parallel y_{\kappa^*} \parallel \mathcal{R}) \in \mathbb{Z}_p^*$.
- Compute the individual SNIZK proof $\pi_{\kappa^*} = c_{\kappa^*} + h_{\kappa^*} \sigma_{\kappa^*}$.
- Aggregate all proofs into a single short proof $\hat{\pi} = \sum_{\kappa^*=1}^{L^*} \pi_{\kappa^*}$.
- Output the aggregated proof pair $(\mathcal{R}, \hat{\pi})$ and send it to the DU.

(b) **DU (Verifier):**

- For each κ^* , compute $h'_{\kappa^*} = H_1(g \parallel y_{\kappa^*} \parallel \mathcal{R})$.
- Accept the proof if and only if the following equation holds:

$$\mathcal{R} \stackrel{?}{=} g^{\hat{\pi}} \cdot \prod_{\kappa^*=1}^{L^*} y_{\kappa^*}^{-h'_{\kappa^*}} \quad (10)$$

If verification succeeds (i.e., outputs $\checkmark$), the DU confirms that the CSP has returned a correct and complete set of matched files; otherwise, it outputs $\perp$. This guarantees result integrity and completeness even under a semi-trusted CSP.

5.7. Outsourced decryption

(14) $\text{PartialDecrypt}(GP, CT_\Phi, SK_{uid}) \rightarrow \{(tpk, tsk), PCT_\Phi\}$

The DU samples $\vartheta \xleftarrow{\$} \mathbb{Z}_p^*$ and sets the outsourced decryption key pair as $tsk = \vartheta$, $tpk = g^\vartheta$. The ES then performs partial decryption and computes an intermediate component $\mathcal{Z}$ as follows, returning the partially decrypted ciphertext $PCT_\Phi = \{\mathcal{Z}, C\}$ to the DU:

$$\begin{aligned} \mathcal{Z} &= \prod_{x \in S} \left(\frac{C_{3,x}^{K_3} \cdot \hat{e}(C_{4,x}, tpk)}{\hat{e}(C_{1,x}, K_1) \cdot \hat{e}(C_{2,x}, K_{2,x})} \right)^{\omega_x} \\ &= \prod_{x \in S} \left(\hat{e}(g, g)^{\mu_{\lambda_x} \vartheta} \right)^{\omega_x} \\ &= \hat{e}(g, g)^{\mu_{\theta_s}} \end{aligned} \quad (11)$$

Algorithm 1 RVSC

Require: Global public parameters GP , search result list SRL , matched files' signatures $\{\sigma_{\kappa^*}\}$

Ensure: Verification output ($\checkmark$ or $\perp$).

```

1:  $\{\kappa^*\}_{1 \leq \kappa^* \leq L^*} \leftarrow SRL$  ▷ Indices of matched files from Search

2: CSP (Prover):
3: Initialize  $\mathcal{R} \leftarrow 1$ ,  $\hat{\pi} \leftarrow 0$ 
4: for  $\kappa^* \leftarrow 1$  to  $L^*$  do
5:    $c_{\kappa^*} \xleftarrow{\$} \mathbb{Z}_p^*$ 
6:    $\mathcal{R} \leftarrow \mathcal{R} \cdot g^{c_{\kappa^*}}$ 
7:    $y_{\kappa^*} \leftarrow g^{\sigma_{\kappa^*}}$ 
8:    $h_{\kappa^*} \leftarrow H_1(g \parallel y_{\kappa^*} \parallel \mathcal{R})$ 
9:    $\pi_{\kappa^*} \leftarrow c_{\kappa^*} + h_{\kappa^*} \sigma_{\kappa^*}$ 
10:   $\hat{\pi} \leftarrow \hat{\pi} + \pi_{\kappa^*}$  ▷ Aggregate all proof into a single one
11: end for

12: DU (Verifier):
13:  $LHS \leftarrow \mathcal{R}$  ▷ Eq. (10), left-hand side
14:  $RHS \leftarrow g^{\hat{\pi}}$  ▷ Eq. (10), right-hand side
15: for  $\kappa^* \leftarrow 1$  to  $L^*$  do
16:    $y_{\kappa^*} \leftarrow g^{\sigma_{\kappa^*}}$ 
17:    $h'_{\kappa^*} \leftarrow H_1(g \parallel y_{\kappa^*} \parallel \mathcal{R})$ 
18:    $RHS \leftarrow RHS \cdot y_{\kappa^*}^{-h'_{\kappa^*}}$ 
19: end for
20: if  $LHS = RHS$  then ▷ Aggregated proof is valid
21:   return  $\checkmark$ 
22: else
23:   return  $\perp$ 
24: end if

```

(15) $\text{FinalDecrypt}(GP, PCT_\Phi, VPK_{ver}, tsk, CT_{F_{ver}^*}) \rightarrow F_{ver}^*$

Upon receiving PCT_Φ , the DU employs the outsourced decryption secret key tsk to recover the access token Φ :

$$\frac{C}{Z^{1/tsk}} = \frac{\Phi \cdot \hat{e}(g, g)^{\mu_s}}{[\hat{e}(g, g)^{\mu_{\theta s}}]^{\frac{1}{\theta}}} = \Phi. \quad (12)$$

With Φ at hand, the DU derives the re-encryption key as $K'_{ver} = KDF(\Phi \parallel H_1(g^{\delta_{ver}}))$, where $g^{\delta_{ver}} \in VPK_{ver}$ is publicly available. The resulting K'_{ver} is used to decrypt the file key K , which in turn decrypts the ciphertext $CT_{F_{ver}^*}$ to recover the target files F_{ver}^* .

5.8. Revocation mechanism

To enhance the practicality of BAMKS, we introduce a revocation mechanism supporting both *attribute revocation* and *user revocation*. This ensures that compromised attributes or revoked users cannot decrypt either existing and future ciphertexts without re-encrypting already stored ciphertexts.

5.8.1. Attribute revocation

(16a) $\text{KeyUpdate}(GP, SK_{uid}, \Gamma_{upd}) \rightarrow \overline{SK}_{uid}$

Upon receiving an attribute revocation message from the KGC, the responsible authority AA_θ selects a random $\bar{\alpha}_\theta \xleftarrow{\$} \mathbb{Z}_p^*$ such that $\bar{\alpha}_\theta \neq \alpha_\theta$, and derives the update factor $\Gamma_{upd} = \frac{\bar{\alpha}_\theta}{\alpha_\theta}$. For each attribute att_i under AA_θ , the user key component is updated as:

$$\bar{K}_{2,i} = K_{2,i}^{\Gamma_{upd}} = H(uid)^{\theta_i} \cdot \text{dpk}_{ver}^{\bar{\alpha}_i} \quad (13)$$

The refreshed user secret key becomes: $\overline{SK}_{uid} = (K_1, \{\bar{K}_{2,i}\}_{i \in S_{uid}}, K_3)$.

(17a) $\text{CTUpdate}(GP, CT_\Phi, \Gamma_{upd}) \rightarrow \overline{CT}_\Phi$

To maintain forward security, the ciphertext component corresponding to the revoked attribute is synchronously updated:

$$\bar{C}_{3,x} = C_{3,x}^{\Gamma_{upd}} = \hat{e}(g, g)^{\frac{\alpha_{\rho(x)} r_x \cdot \text{dsk}_{ver}}{h_{\varpi}}}. \quad (14)$$

The updated ciphertext becomes: $\overline{CT}_\Phi = (C, C_0, \{C_{1,x}, C_{2,x}, \bar{C}_{3,x}, C_{4,x}\}_{x \in [1, \ell]})$.

5.8.2. User revocation

(16b) $\text{KeyUpdate}(GP, SK_{uid}, \Gamma_{upd}) \rightarrow \overline{SK}_{uid}$

When a DU is revoked, the RTA updates the user registration list to $\text{RegList}'$ and refreshes the membership control factor as $\bar{h}_{\varpi'} = H_1(\varpi')$, where $\varpi' \xleftarrow{\$} \mathbb{Z}_p^*$ and $\varpi' \neq \varpi$. The resulting update factor is defined as: $\Gamma_{upd} = \frac{h_{\varpi}}{h_{\varpi'}}$. Each non-revoked user locally updates its key component as:

$$\bar{K}_3 = K_3 / \Gamma_{upd} = \bar{h}_{\varpi'}. \quad (15)$$

Thus, the refreshed user secret key becomes: $\overline{SK}_{uid} = (K_1, \{K_{2,i}\}_{i \in S_{uid}}, \bar{K}_3)$.

(17b) $\text{CTUpdate}(GP, CT_\Phi, \Gamma_{upd}) \rightarrow \overline{CT}_\Phi$

Given Γ_{upd} , the ciphertext component is updated as:

$$\bar{C}_{3,x} = C_{3,x}^{\Gamma_{upd}} = \hat{e}(g, g)^{\frac{\alpha_{\rho(x)} r_x \cdot \text{dsk}_{ver}}{h_{\varpi'}}}. \quad (16)$$

Thus, $\overline{CT}_\Phi = (C, C_0, \{C_{1,x}, C_{2,x}, \bar{C}_{3,x}, C_{4,x}\}_{x \in [1, \ell]})$.

Remark 5. Neither revocation procedure requires a full re-encryption of Φ . Only $C_{3,x}$ is refreshed, while $C, C_0, C_{1,x}, C_{2,x}$, and $C_{4,x}$ remain unchanged. This ensures efficient ciphertext refresh and forward security, since revoked users or attributes cannot decrypt updated ciphertexts.

5.9. User traceability

If a DU's decryption key is leaked or misused, the system must first identify the misbehaving user prior to revocation. Since each key in $\text{KeyGen}(\cdot)$ is bound to a trace credential $\mathcal{T}_{uid}$ stored in TraceList , the RTA proceeds as follows.

(18) $\text{KeySanityCheck}(\widetilde{SK}_{uid}) \rightarrow \{0, 1\}$

Given a suspected leaked key $\widetilde{SK}_{uid}$, the RTA computes $\widetilde{\mathcal{T}} = \hat{e}((\widetilde{K}_1)^{-1}, g)$ and checks whether $\widetilde{\mathcal{T}}$ appears in TraceList . If so, output 1 (valid key); otherwise output 0.

(19) $\text{Trace}(\widetilde{SK}_{uid}) \rightarrow uid$

If $\text{KeySanityCheck}(\widetilde{SK}_{uid}) = 1$, the RTA identifies the misbehaving user by $uid = \text{TraceList}[\widetilde{\mathcal{T}}]$. The identified uid is then inserted into RegList , and the corresponding $\text{KeyUpdate}(\cdot)$ and $\text{CTUpdate}(\cdot)$ procedures are triggered.

Remark 6. This traceability mechanism is non-invasive: it does not alter the structure of user keys and ciphertexts. It guarantees that any DU who leaks or misuses its keys can be non-repudially identified and revoked.

5.10. Index updating

(20) $\text{IndexUpdate}(GP, \text{Index}_{\mathcal{W}}) \rightarrow \text{Index}'_{\mathcal{W}}$

To support dynamic file and keyword evolution, the system allows refreshing the encrypted index without re-encrypting data files. Given the updated file-set master key δ'_{ver} and keyword-set master key ξ'_{ver} , the DO computes the index update factor: $X_{upd} = g^{\Delta}$, where $\Delta = (\delta'_{ver} - \delta_{ver}) / (\xi'_{ver} - \xi_{ver}) \in \mathbb{Z}_p^*$.

Each index component is then updated as:

$$\begin{aligned} I'_1 &= g^{\eta+s} \cdot X_{upd}, \\ I'_{2,r} &= g^{\gamma\eta_r} \cdot X_{upd}, \\ I'_{\kappa,j} &= g^{\eta H_1(w_j)} \cdot X_{upd}, \end{aligned} \quad (17)$$

$$\text{Index}'_{\mathcal{W}} = \left(I'_1, \{I'_{2,r}\}_{r \in [1,N]}, \{I'_{\kappa,j}\}_{\kappa \in [1,L'], j \in [1,m']} \right).$$

A valid search requires the trapdoor and index to belong to the same version identifier ver (implicitly carried in $g^{\delta_{ver}}$). After an update to ver' , the exponent offset Δ breaks cross-version compatibility, so trapdoors generated under ver will fail on $\text{Index}'_{\mathcal{W}}$ by design. Hence, DUs only need to re-generate trapdoors for the current version; the $\text{Search}(\cdot)$ algorithm remains unchanged.

Remark 7. The index update mechanism provably ensures both forward and backward search security: a trapdoor generated under version ver cannot match an index from version ver' , and vice versa, except with negligible probability. This follows from the hardness of the discrete logarithm problem in $\mathbb{G}$ and the random oracle assumption on H_1 .

6. Correctness and security analysis

6.1. Correctness proof

6.1.1. Computational correctness of the keyword search

$$\begin{aligned} \frac{\hat{e}\left(\prod_{r=1}^N I_{2,r}, T_2\right)}{\hat{e}(I_1/C_0, T_1)} &= \frac{\hat{e}\left(\prod_{r=1}^N g^{\gamma\eta_r}, g^{\phi+\frac{\chi}{Z}}\right)}{\hat{e}\left(\frac{g^{\eta+s}}{g^s}, g^{\gamma\phi}\right)} \\ &= \frac{\hat{e}\left(g^{\gamma \sum_r \eta_r}, g^{\phi}\right) \cdot \hat{e}\left(g^{\gamma \sum_r \eta_r}, g^{\frac{\chi}{Z}}\right)}{\hat{e}(g^{\eta}, g^{\gamma\phi})} \\ &= \frac{\hat{e}(g, g)^{\eta\phi\gamma} \cdot \hat{e}(g, g)^{\frac{\eta\chi\gamma}{Z}}}{\hat{e}(g, g)^{\eta\phi\gamma}} \\ &= \hat{e}(g, g)^{\frac{\eta\chi\gamma}{Z}} \end{aligned} \quad (18)$$

$$\begin{aligned} \hat{e}\left(\prod_{j^*=1}^{m^*} I_{\kappa,j^*}, T_3\right) &= \hat{e}\left(\prod_{j^*=1}^{m^*} g^{\eta H_1(\tilde{w}_{j^*})}, (g^{\gamma})^{\frac{\chi}{Z \sum_{j^*=1}^{m^*} H_1(\tilde{w}_{j^*})}}\right) \\ &= \hat{e}\left(g^{\eta \sum_{j^*=1}^{m^*} H_1(\tilde{w}_{j^*})}, g^{\frac{\chi\gamma}{Z \sum_{j^*=1}^{m^*} H_1(\tilde{w}_{j^*})}}\right) \\ &= \hat{e}(g, g)^{\frac{\eta\chi\gamma}{Z}} \end{aligned} \quad (19)$$

If the underlying index keywords $\widetilde{\mathcal{W}}$ in $\text{Index}_{\mathcal{W}}$ is identical to the query keywords $\widetilde{\mathcal{W}}$ in $\text{Trap}_{\widetilde{\mathcal{W}}}$, then

$$\frac{\hat{e}\left(\prod_{r=1}^N I_{2,r}, T_2\right)}{\hat{e}(I_1/C_0, T_1)} = \hat{e}\left(\prod_{j^*=1}^{m^*} I_{\kappa,j^*}, T_3\right) \quad (20)$$

6.1.2. Computational correctness of the result verification

$$\begin{aligned}
\mathcal{R}' &= g^{\hat{z}} \cdot \prod_{k^*=1}^{L^*} y_k^{(-h'_{k^*})} \\
&= g^{\sum_{k^*=1}^{L^*} (c_{k^*} + h_{k^*} \sigma_{k^*})} \cdot \prod_{k^*=1}^{L^*} g^{\sigma_{k^*} (-h'_{k^*})} \\
&= g^{\sum_{k^*=1}^{L^*} c_{k^*}} \cdot g^{\sum_{k^*=1}^{L^*} h_{k^*} \sigma_{k^*}} \cdot g^{(-\sum_{k^*=1}^{L^*} \sigma_{k^*} h'_{k^*})} \\
&= g^{\sum_{k^*=1}^{L^*} c_{k^*}} \\
&= \mathcal{R}
\end{aligned} \tag{21}$$

6.1.3. Computational correctness of the partial decryption

$$\begin{aligned}
\mathcal{Z}' &= \frac{C_{3,x}^{K_3} \cdot \hat{e}(C_{4,x}, tpk)}{\hat{e}(C_{1,x}, K_1) \cdot \hat{e}(C_{2,x}, K_{2,x})} \\
&= \frac{\hat{e}(g, g)^{\frac{\alpha_{\rho(x)} r_x \delta_{ver}}{\epsilon h_w}} \cdot h_w \hat{e}(g^{\mu \lambda_x}, g^\theta)}{\hat{e}(g^{\beta_{\rho(x)} r_x}, H(uid)^{-t}) \hat{e}(g^{r_x}, H(uid)^{t \beta_{\rho(x)}} g^{\frac{\delta_{ver} \alpha_{\rho(x)}}{\epsilon}})} \\
&= \frac{\hat{e}(g, g)^{\frac{\alpha_{\rho(x)} r_x \delta_{ver}}{\epsilon}} \hat{e}(g, g)^{\mu \lambda_x \theta}}{\hat{e}(g, H(uid))^{-t \beta_{\rho(x)} r_x} \hat{e}(g, H(uid))^{t \beta_{\rho(x)} r_x} \hat{e}(g, g)^{\frac{\alpha_{\rho(x)} r_x \delta_{ver}}{\epsilon}}}} \\
&= \hat{e}(g, g)^{\mu \lambda_x \theta}
\end{aligned} \tag{22}$$

Since a set of constants $\{\omega_x\}_{x \in S}$ satisfying $\sum_x \lambda_x \omega_x = s$, we get the following simplified $\mathcal{Z}$:

$$\mathcal{Z} = \prod_{x \in S} (\hat{e}(g, g)^{\mu \lambda_x \theta})^{\omega_x} = \hat{e}(g, g)^{\mu \theta \sum_x \lambda_x \omega_x} = \hat{e}(g, g)^{\mu \theta s} \tag{23}$$

6.2. Security proof

In this section, we present a comprehensive security analysis of BAMKS, covering ciphertext confidentiality (IND-CPA; see [Section 6.2.1](#)), keyword search privacy (IND-CKA; see [Section 6.2.2](#)), decryption security (see [Section 6.2.3](#)), and search result verification (see [Section 6.2.4](#)). Our proofs proceed under the Decisional Bilinear Diffie-Hellman (DBDH) and discrete-logarithm (DLP) assumptions, assuming that the hash functions are modeled as random oracles.

6.2.1. IND-CPA security proof

Theorem 1. *If a PPT adversary $\mathcal{A}$ breaks the IND-CPA security of BAMKS with non-negligible advantage $\epsilon > 0$, then there exists a PPT simulator $\mathcal{B}$ that distinguishes a DBDH tuple from random with advantage at least $\epsilon/2$.*

Proof. Consider a security game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. We construct a PPT simulator $\mathcal{B}$ that uses $\mathcal{A}$ to solve the DBDH problem with advantage at least $\epsilon/2$. Let $(p, g, \hat{e}, \mathbb{G}, \mathbb{G}_T)$ be bilinear parameters generated by the DBDH challenger $\mathcal{C}$. The challenger randomly selects $a_1, a_2, a_3 \xleftarrow{\$} \mathbb{Z}_p^*$ and flips a coin $b \in \{0, 1\}$, and gives to $\mathcal{B}$ the challenge tuple $(g^{a_1}, g^{a_2}, g^{a_3}, Q)$, where $Q = \hat{e}(g, g)^{a_1 a_2 a_3}$ if $b = 0$; and $Q \xleftarrow{\$} \mathbb{G}_T$ if $b = 1$. The task of $\mathcal{B}$ is to determine whether $\hat{e}(g, g)^{a_1 a_2 a_3} \stackrel{?}{=} Q$, given that a_1, a_2, a_3 and Q remain unknown to $\mathcal{B}$. The simulator $\mathcal{B}$ then interacts with $\mathcal{A}$ as the challenger in the following game.

- **Initialization:** $\mathcal{A}$ corrupts a subset of authorities $C_\theta \subseteq U_\theta$ (leaving at least two honest), reveals C_θ to $\mathcal{B}$, and submits a challenge access structure $\mathbb{A}^* = (\mathbb{M}_{\ell^* \times n^*}^*, \rho^*)$ to $\mathcal{B}$, where attributes may be correspond to both corrupted and honest authorities.
- **Setup:** Given the DBDH challenge $(g^{a_1}, g^{a_2}, g^{a_3}, Q)$, $\mathcal{B}$ simulates the output of the DKG without actually executing the protocol:
 - Set $g^\mu = g^{a_1}$. Randomly select a global mask $v \xleftarrow{\$} \mathbb{Z}_p^*$ and set $\hat{e}(g, g)^\mu = \hat{e}(g^{a_1}, g^{a_2}) \cdot \hat{e}(g, g)^v = \hat{e}(g, g)^{a_1 a_2} \cdot \hat{e}(g, g)^v$. Randomly choose $\gamma \xleftarrow{\$} \mathbb{Z}_p^*$ (consistent with DKG output). The global parameters are then published as $GP = \{p, g, \hat{e}, \mathbb{G}, \mathbb{G}_T, H, H_1, g^\mu, g^\gamma, \hat{e}(g, g)^\mu\}$.
 - For each honest authority AA_θ , run $\text{AuthSetup}(\cdot)$ to randomly select $\alpha_\theta, \beta_\theta \xleftarrow{\$} \mathbb{Z}_p^*$, set $sk_{AA_\theta} = \{\alpha_\theta, \beta_\theta\}$, and publish $pk_{AA_\theta} = \{\hat{e}(g, g)^{\alpha_\theta}, g^{\beta_\theta}\}$.
 - Run $\text{DUSetup}(\cdot)$ to select $\varpi \xleftarrow{\$} \mathbb{Z}_p^*$ and set $h_w = H_1(\varpi)$. This factor is fixed for all non-revoked users during the game.
 - Run $\text{FileSetup}(\cdot)$ to choose $\delta_{ver}, \epsilon \xleftarrow{\$} \mathbb{Z}_p^*$ and publish $VPK_{ver} = \{g^{\delta_{ver}}, dpk_{ver} = g^{\delta_{ver}/\epsilon}\}$, while keeping $VMK_{ver} = \{\delta_{ver}, \epsilon\}$ secret.

- **Query Phase 1:** $\mathcal{A}$ adaptively request user secret keys for (uid, S_{uid}) under the standard restriction $S_{uid} \cup S_{corr} \not\models \mathbb{A}^*$, where S_{corr} collects attributes managed by corrupted authorities. For each valid query, $\mathcal{B}$ executes $\text{KeyGen}(\cdot)$ as follows: choose $t \xleftarrow{\$} \mathbb{Z}_p^*$, set $K_1 = H(uid)^{-t}$, and for every attribute $\text{att}_i \in S_{uid}$ managed by AA_θ set $K_{2,i} = H(uid)^{t\beta_\theta} \cdot \text{dpk}_{ver}^{a_\theta} = H(uid)^{t\beta_\theta} \cdot g^{\delta_{ver}a_\theta/\epsilon}$. Return $SK_{uid} = (K_1, \{K_{2,i}\}_{i \in S_{uid}}, K_3)$, where $K_3 := \hat{h}_w$. Note that μ does not appear in $\text{KeyGen}(\cdot)$, and $\hat{h}_w$ is randomly selected independently of the DBDH instance; hence, the distribution is identical to the real system.
- **Challenge:** $\mathcal{A}$ submits two equal-length messages (m_0, m_1) for the challenge policy $\mathbb{A}^*$. $\mathcal{B}$ flips a coin $\tau \in \{0, 1\}$ and forms a challenge ciphertext following the real encryption, except for the message component.
 - Set $s = a_3$ (thus $C_0 = g^s = g^{a_3}$) and selects $\vec{v} = (s, v_2, \dots, v_n)^T$ to compute LSSS shares $\lambda_x = \mathbb{M}_x^* \cdot \vec{v}$.
 - For each row x , randomly select $r_x \xleftarrow{\$} \mathbb{Z}_p^*$ and run $\text{TokenEnc}_{off-chain}(\cdot)$ and $\text{TokenEnc}_{on-chain}(\cdot)$ to compute $C_{1,x} = g^{\beta_{\rho(x)}r_x}$, $C_{2,x} = g^{r_x}$, $C_{3,x} = \hat{e}(g, g)^{\frac{a_{\rho(x)}r_x\delta_{ver}}{\epsilon h_w}}$, and $C_{4,x} = (g^\mu)^{\lambda_x} = (g^{a_1})^{\lambda_x}$.
 - Define the access token component $C = m_\tau \cdot Q \cdot \hat{e}(g, g)^{vs}$, and output $CT_\Phi^* = ((\mathbb{M}^*, \rho^*), C, C_0, \{C_{1,x}, C_{2,x}, C_{3,x}, C_{4,x}\}_x)$.
 - If $b = 0$, then $C = m_\tau \cdot \hat{e}(g, g)^{a_1a_2a_3} \cdot \hat{e}(g, g)^{vs} = m_\tau \cdot \hat{e}(g, g)^{(a_1a_2+vs)} = m_\tau \cdot \hat{e}(g, g)^{\mu s}$, so CT_Φ^* is identically distributed to a real ciphertext.
 - If $b = 1$, then $C = m_\tau \cdot Q \cdot \hat{e}(g, g)^{vs}$ with $Q \xleftarrow{\$} \mathbb{G}_T$, where Q acts as a multiplicative one-time pad in $\mathbb{G}_T$, thereby perfectly hiding m_τ from any CPA adversary.
- **Query Phase 2:** Same as **Phase 1** under the same restriction $\neg(S_{uid} \models \mathbb{A}^*)$.
- **Guess:** $\mathcal{A}$ outputs τ' . $\mathcal{B}$ sets its DBDH guess $b' = 0$ if $\tau' = \tau$, and $b' = 1$ otherwise.

Let $p_0 = \Pr[\tau' = \tau \mid b = 0]$ and $p_1 = \Pr[\tau' = \tau \mid b = 1]$. From the above, $p_0 = \frac{1}{2} + \epsilon$ and $p_1 = \frac{1}{2}$. Hence,

$$\Pr[b' = b] = \frac{1}{2}p_0 + \frac{1}{2}p_1 = \frac{1}{2} + \frac{\epsilon}{2}, \quad \text{and} \quad \text{Adv}_B^{\text{DBDH}} = \left| \Pr[b' = b] - \frac{1}{2} \right| = \frac{\epsilon}{2}.$$

Therefore any PPT adversary $\mathcal{A}$ that breaks the IND-CPA security of BAMKS with advantage ϵ yields a PPT distinguisher for DBDH with advantage at least $\epsilon/2$.

In conclusion, $\mathcal{B}$'s advantage to win the above security game is, where $Q \xleftarrow{\$} \mathbb{G}_T$:

$$\text{Adv}(\mathcal{B}) = \frac{1}{2} \left(\Pr[\mathcal{B}(g, g^{a_1}, g^{a_2}, g^{a_3}, \hat{e}(g, g)^{a_1a_2a_3}) = 0] + \Pr[\mathcal{B}(g, g^{a_1}, g^{a_2}, g^{a_3}, Q) = 0] \right) - \frac{1}{2} = \frac{\epsilon}{2}.$$

6.2.2. IND-CKA security proof

Theorem 2. If a PPT adversary $\mathcal{A}$ breaks the IND-CKA security of BAMKS with non-negligible advantage $\epsilon > 0$, then there exists a PPT simulator $\mathcal{B}$ that distinguishes a DBDH tuple from random with advantage at least $\epsilon/2$.

Proof. Similar to the security game in [Theorem 1](#), we construct a simulator $\mathcal{B}$ that solves the DBDH problem if an adversary $\mathcal{A}$ can break the CKA game. Let $(p, g, \hat{e}, \mathbb{G}, \mathbb{G}_T)$ be bilinear parameters. The DBDH challenger $\mathcal{C}$ randomly selects $a_1, a_2, a_3 \in \mathbb{Z}_p^*$ and flip a coin $b \in \{0, 1\}$, and hands to $\mathcal{B}$ the tuple $(g^{a_1}, g^{a_2}, g^{a_3}, Q)$, where $Q = \hat{e}(g, g)^{a_1a_2a_3}$ if $b = 0$ and $Q \xleftarrow{\$} \mathbb{G}_T$ if $b = 1$. Acting as the CKA challenger, $\mathcal{B}$ interacts with $\mathcal{A}$ and attempts to distinguish these two cases in the following security game:

- **Setup:** Given the DBDH challenge $(g^{a_1}, g^{a_2}, g^{a_3}, Q)$, $\mathcal{B}$ programs the public parameters similarly to [Theorem 1](#):
 - Set $g^\mu = g^{a_1}$. Let $v \xleftarrow{\$} \mathbb{Z}_p^*$ be a random mask, and publish $\hat{e}(g, g)^\mu = \hat{e}(g, g)^{a_1} \cdot \hat{e}(g, g)^v = \hat{e}(g, g)^{a_1a_2+vs}$. Then run $\text{GlobalSetup}(\cdot)$ to randomly select $\gamma \xleftarrow{\$} \mathbb{Z}_p^*$ and obtain g^γ . Generate the authority key pairs $(pk_{AA_\theta}, sk_{AA_\theta})$ and the file version keys (VPK_{ver}, VMK_{ver}) , and incorporate them in the global parameters GP .
 - Run $\text{UserSetup}(\cdot)$ to randomly select $\chi, z \xleftarrow{\$} \mathbb{Z}_p^*$ and set $usk = \{\chi, z\}$, $upk = \{g^\chi, g^z\}$.
- **Query Phase 1:** $\mathcal{A}$ adaptively issues trapdoor queries for the user-chosen keywords $\tilde{\mathcal{W}}$. For each valid query, $\mathcal{B}$ runs $\text{TrapGen}(\cdot)$: randomly select $\phi \xleftarrow{\$} \mathbb{Z}_p^*$ and compute $T_1 = g^{\gamma\phi}$, $T_2 = g^{\phi+\chi/z}$, $T_3 = (g^\gamma)^{\chi/(z \sum_j H_1(\tilde{w}_j))}$, where $\{\tilde{w}_j\} \subseteq \tilde{\mathcal{W}}$. Thus, output the tapdoor $\text{Trap}_{\tilde{\mathcal{W}}} = \{T_1, T_2, T_3\}$. We impose the standard CKA restriction that no oracle query may enable a trivial test against the challenge index.
- **Challenge:** $\mathcal{A}$ submits two keyword dictionaries $\mathcal{W}_0, \mathcal{W}_1$ of equal size that were not previously queried. $\mathcal{B}$ flips a coin $\tau \in \{0, 1\}$ and must return an encrypted index $\text{Index}_{\mathcal{W}_\tau}$ together with the on-chain ciphertext C_0 . Specifically, $\mathcal{B}$ acts as follows:
 - Set $\eta = a_2$ and choose random shares $\{\eta_r\}$ summing to η ; run $\text{TokenEnc}_{on-chain}(\cdot)$ to set $s = a_3$ (so $C_0 = g^s = g^{a_3}$).
 - Run $\text{IndexGen}(\cdot)$ to compute the index components: $I_1 = g^{\eta+s} = g^{a_2} \cdot g^{a_3}$, $I_{2,r} = (g^\gamma)^{\eta_r}$, $I_{\kappa,j} = (g^\eta)^{H_1(w_j)} = (g^{a_2})^{H_1(w_j)}$. If $b = 0$ (real DBDH), output the index $\text{Index}_{\mathcal{W}_\tau}$ built with the above $(I_1, I_{2,r}, I_{\kappa,j})$ and $C_0 = g^{a_3}$. If $b = 1$ (random DBDH), output a shape-identical but random index: resample $I_1 \xleftarrow{\$} \mathbb{G}$, $I_{2,r} \xleftarrow{\$} \mathbb{G}$ for all r , and $I_{\kappa,j} \xleftarrow{\$} \mathbb{G}$ for all κ, j , keeping $C_0 = g^{a_3}$.
- **Query Phase 2.** Identical to **Phase 1** under the same admissibility restriction; $\mathcal{B}$ keeps answering $\text{TrapGen}(\cdot)$ queries honestly.
- **Guess.** $\mathcal{A}$ outputs a bit τ' . $\mathcal{B}$ sets its DBDH guess $b' = 0$ if $\tau' = \tau$ and $b' = 1$ otherwise.

When $b = 0$, the challenge index and all oracle answers are distributed exactly as in the real system, and thus $\Pr[\tau' = \tau \mid b = 0] = \frac{1}{2} + \epsilon$. When $b = 1$, the challenge index is independent of $\mathcal{W}_\tau$, and by admissibility $\mathcal{A}$ cannot invoke $\text{Search}(\cdot)$ on the challenge index;

Table 2
Functionality comparison.

Scheme	F_1	F_2	F_3	F_4	F_5	F_6
BLUMA [8]	×	✓	×	×	×	×
MKSV-ABE [21]	×	×	✓	×	✓	×
EMK-ABSE [22]	×	×	×	✓	×	×
BA-RMKABSE [33]	×	✓	×	✓	✓	×
BC-dPEKS [40]	×	✓	×	✓	×	×
VMSE [54]	×	×	×	✓	✓	×
Ours	✓	✓	✓	✓	✓	✓

Note: F_1 : dynamic data sharing, F_2 : blockchain, F_3 : multi-owner, F_4 : multi-keyword search, F_5 : results verification, F_6 : revocation & traceability; “✓” means support; “×” means not supported.

Table 3
Theoretical comparison of computational cost in ABE.

Scheme	KeyGen	Encrypt	Decrypt	
			Outsourced	DU
BLUMA [8]	$(4 I + 1)E + I M$	$(\mathcal{L} + 1)E_T + 5 \mathcal{L} E + M$	$4 I P + I E_T + 4 I M$	$E_T + M$
MKSV-ABE [21]	$(2 I + 4)E + M$	$E_T + (1 + \mathcal{L} + N)E + (\mathcal{L} + 1)M$	$(I + 2)P + I E_T + N E + 2 I M$	M
EMK-ABSE [22]	$(2 I + 8)E + 2M$	$E_T + (3 \mathcal{L} + 1)E + (\mathcal{L} + 1)M$	$(2 I + 1)P + I E_T + 2 I M$	$P + 2E_T + M$
BA-RMKABSE [33]	$(2 I + 2)E + I M$	$E_T + (4 \mathcal{L} + 2)E + (\mathcal{L} + 1)M$	$3 I P + (I + 1)E_T + 2 I M$	E_T
Ours	$(2 I + 1)E$	$(\mathcal{L} + 1)E_T + (3 \mathcal{L} + 1)E$	$3 I P + (I + 1)E_T + 4 I M$	E_T

Note: $|I|$: attribute number within user secret keys; $|\mathcal{L}|$: attribute number within access structure; P : bilinear pairing operation on $\mathbb{G}$; E_T , E : exponentiation operation on $\mathbb{G}_T$ and $\mathbb{G}$; M : multiplication operation on $\mathbb{G}_T$ and $\mathbb{G}$.

Table 4
Theoretical comparison of computational cost in ABKS.

Scheme	IndexGen	TrapGen	Search
EMK-ABSE [22]	$(m + 3)E$	$(m^* + 2)E + m^* M$	$4P + (m^* + 2)M$
BA-RMKABSE [33]	$(m + 3)E$	$3E$	$3P + (m^* + 2)M$
BC-dPEKS [40]	$(m + 1)P + 3E + m M$	$(m^* + 2)E + (m^* + 1)M$	$2P + 2E + (2 m^* + 1)M$
VMSE [54]	$(m + 1)E$	$(m^* + 2)E + m^* M$	$3P + (m^* + 2)M$
Ours	$(m + N + 1)E$	$3E$	$3P + (m^* + N)M$

Note: $|U|$: number of attributes in system attribute universes; $|N|$: number of data owners; $|m|$: number of keywords in the keyword dictionary; $|m^*|$: number of keywords in the query keyword set.

hence $\Pr[\tau' = \tau \mid b = 1] = \frac{1}{2}$. Therefore,

$$\Pr[b' = b] = \frac{1}{2}(\frac{1}{2} + \epsilon) + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2} + \frac{\epsilon}{2}, \quad \text{and} \quad Adv_B^{\text{DBDH}} = \left| \Pr[b' = b] - \frac{1}{2} \right| = \frac{\epsilon}{2}. \quad (24)$$

Hence, any PPT adversary $\mathcal{A}$ that breaks IND-CKA for BAMKS with advantage ϵ yields a PPT distinguisher for DBDH with advantage at least $\epsilon/2$.

Remark 8. As in standard adaptive CKA security models, trapdoor queries in Phases 1 and 2 are restricted so that the adversary cannot issue a query that would cause $\text{Search}(GP, C_0, \text{Index}_{\mathcal{W}_\tau}, \text{Trap}_{\mathcal{W}})$ to accept on the challenge index (e.g., exact match, subset or superset), which prevents trivial wins while preserving general querying capability.

6.2.3. Decryption security proof

Theorem 3. Assume DLP is hard in $\mathbb{G}$, $\mathbb{G}_T$, H , H_1 are random oracles, the KDF is a PRF, and the symmetric encryption is IND-CPA secure. Then for any PPT adversary: (i) Any user whose attributes do not satisfy the access policy cannot recover the access token Φ (and hence cannot decrypt), even if colluding with corrupted authorities. (ii) A semi-trusted ES performing partial decryption learns nothing about Φ or the file encryption key K ; moreover, the file ciphertext is indistinguishable from random to any party that cannot reconstruct Φ .

Proof.

(i) Unauthorized user decryption resistance.

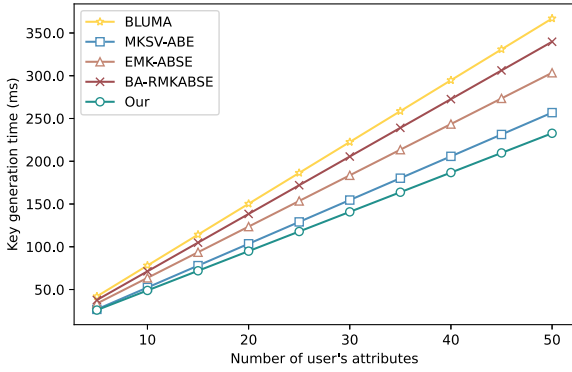
By the LSSS property, if the user's attributes do not satisfy the policy, then there exist no weights $\{\omega_x\}$ that reconstruct $\sum_x \omega_x \lambda_x = s$. Consequently, the ES aggregate $\mathcal{Z}$ (cf. Eq. (11)) cannot equal the target $\hat{e}(g, g)^{\mu^{\theta s}}$. Hence, even after the DU removes ϑ using tsk (i.e.,

Table 5

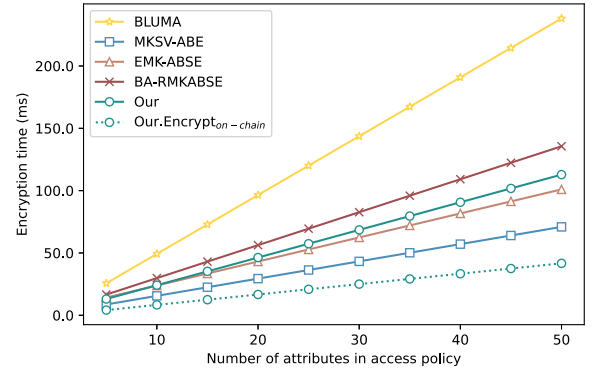
Theoretical comparison of storage overhead.

Scheme	KeySZ	CipherSZ	IndexSZ	TrapSZ
BLUMA [8]	$(3 S + 1) G + Z_p $	$(\mathcal{L} + 1) G_T + 5 \mathcal{L} G $	–	–
MKSV-ABE [21]	$(2 S + 3) G + Z_p^* $	$ G_T + (\mathcal{L} + 1) G $	–	–
EMK-ABSE [22]	$(S + 6) G + Z_p^* $	$ G_T + (2 \mathcal{L} + 1) G $	$(m + 3) G $	$4 G $
BA-RMKABSE [33]	$(2 S + 1) G $	$ G_T + (3 \mathcal{L} + 2) G $	$(m + 3) G $	$3 G $
BC-dPEKS [40]	–	–	$ m G_T + G $	$2 G $
VMSE [54]	–	–	$(m + 1) G $	$3 G $
Ours	$(S + 1) G + Z_p^* $	$(\mathcal{L} + 1) G_T + (3 \mathcal{L} + 1) G $	$(m + N + 1) G $	$3 G $

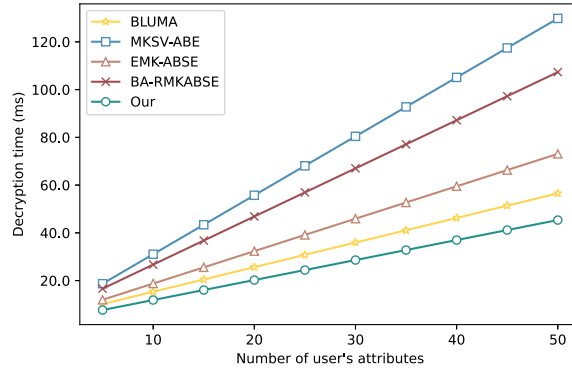
Note: KeySZ: size of user secret keys, CipherSZ: size of on- and off-chain ciphertext of access token, IndexSZ: size of index, TrapSZ: size of trapdoor. $|S|$: attribute numbers in the user attribute set; $|G_T|$, $|G|$: size of a element in groups G_T and G , $|G_T| = |G| = 72$ bytes; $|Z_p^*|$: size of a element in Z_p^* , $|Z_p^*| = 56$ bytes; “–” means without the corresponding parameter.



(a) KeyGen



(b) Ecnrypt



(c) Decrypt

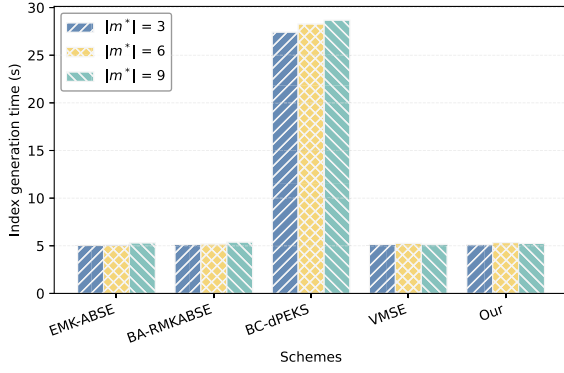
Fig. 2. Comparison of computational overhead in KeyGen, Encrypt and Decrypt algorithms ($|m| = 100$, $|m^*| = 3$).

computes $\frac{C}{Z^{1/isk}}$, the result does not yield Φ ; rather, it is Φ multiplied by a uniform element in G_T , which information-theoretically hides Φ . Therefore, no unauthorized key material enables decryption, even under collusion.

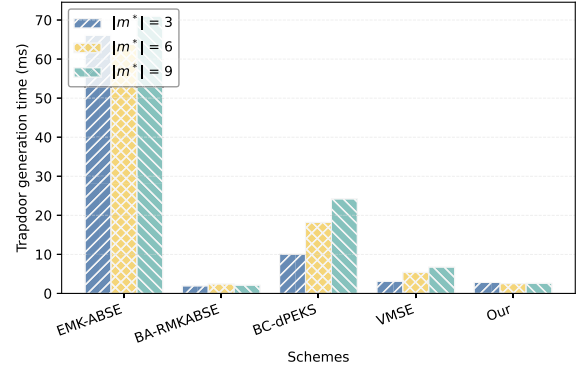
(ii) Outsourced decryption confidentiality.

Access token privacy against the ES. The ES learns only $tpk = g^\theta$ and $Z = \mathcal{E}^{u\theta s}$, where $\mathcal{E} = \hat{e}(g, g)$. From these it can form $\mathcal{E}^\theta = \hat{e}(g, tpk)$, so its view is the pair $(\mathcal{E}^\theta, (\mathcal{E}^\theta)^{us})$. Recovering $\mathcal{E}^{us}$ from this pair requires computing a θ -th root in G_T (i.e., raising to $\theta^{-1} \pmod{p}$), which amounts to inverting the map $x \mapsto x^\theta$ in a prime-order group and is DLP-hard. Hence, under DLP, (Z, tpk) reveals no information about $\mathcal{E}^{us}$ and thus nothing about Φ .

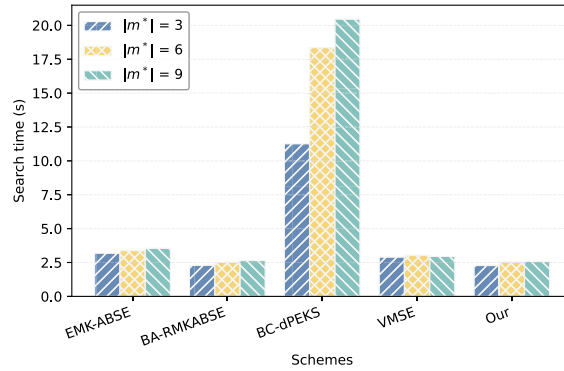
Confidentiality of $\mathcal{K}$. After an authorized DU recovers Φ , it derives $\mathcal{K}'_{ver} = KDF(\Phi \parallel H_1(g^{\delta_{ver}}))$. Under the PRF assumption for KDF and the RO model for H_1 , $\mathcal{K}'_{ver}$ is pseudorandom to any party lacking Φ . Encapsulating $\mathcal{K}$ under $\mathcal{K}'_{ver}$ with an IND-CPA symmetric



(a) IndexGen



(b) TrapGen



(c) Search

Fig. 3. Comparison of computational overhead in IndexGen, TrapGen, Search algorithms ($|m| = 100$, $|m^*| \in \{3, 6, 9\}$).

scheme makes the adversary's view independent of $\mathcal{K}$ unless Φ is known; hence the file ciphertext is indistinguishable from random to any party that cannot reconstruct Φ .

Thus, BAMKS ensures confidentiality: only a DU holding tsk and valid attribute keys can recover Φ , derive $\mathcal{K}$, and decrypt the files; whereas the ES learns nothing about Φ or $\mathcal{K}$.

6.2.4. SNIZK security proof

Theorem 4. Assuming that DLP is hard in $\mathbb{G}$ and H_1 is modeled as a random oracle, the aggregated SNIZK proof used in ProofVerify($\cdot$) satisfies: (i) completeness: all honestly-generated proofs verify; and (ii) soundness: any PPT prover (e.g., the CSP) that convinces the verifier with non-negligible probability must know witnesses $\{\sigma_{k^*}\}$ such that $y_{k^*} = g^{\sigma_{k^*}}$. Hence, producing an accepting proof without valid witnesses would solve DLP.

Proof. Let $y_{k^*} = g^{\sigma_{k^*}}$ be the tags of the matched files. The prover randomly selects $c_{k^*} \xleftarrow{\$} \mathbb{Z}_p^*$, sets $\mathcal{R} = g^{\sum_{k^*=1}^{L^*} c_{k^*}}$, derives challenges $h_{k^*} = H_1(g \| y_{k^*} \| \mathcal{R})$, and outputs the aggregated response $\hat{\pi} = \sum_{k^*=1}^{L^*} (c_{k^*} + h_{k^*} \sigma_{k^*})$. The verifier accepts if Eq. (10) holds.

- (i) **Completeness.** If $y_{k^*} = g^{\sigma_{k^*}}$ and $\hat{\pi} = \sum_{k^*=1}^{L^*} (c_{k^*} + h_{k^*} \sigma_{k^*})$, then $g^{\hat{\pi}} = g^{\sum_{k^*=1}^{L^*} c_{k^*}} \cdot \prod (g^{\sigma_{k^*}})^{h_{k^*}} = \mathcal{R} \cdot \prod y_{k^*}^{h_{k^*}}$, so Eq. (10) holds.
- (ii) **Soundness.** Model H_1 as a random oracle. Suppose a PPT CSP convinces the verifier with non-negligible probability ϵ . Fix the prover's internal randomness so that $\mathcal{R} = g^{\sum_{k^*=1}^{L^*} c_{k^*}}$ is determined. Run the prover once with the random oracle and obtain an accepting transcript $(\mathcal{R}, \hat{\pi}, \{h_{k^*}\})$. Then replay the prover using the same internal randomness and oracle answers, except that for at least one query $(g \| y_{k^*} \| \mathcal{R})$ the oracle outputs an independent value $h'_{k^*} \neq h_{k^*}$. By the generalized forking lemma, with probability $\bar{\epsilon} = \Omega(\epsilon^2 / \text{poly}(q_H, L^*))$ this second run also succeeds, yielding two accepting transcripts for the same $\mathcal{R}$: $g^{\hat{\pi}} = \mathcal{R} \cdot \prod y_{k^*}^{h_{k^*}}$ and $g^{\hat{\pi}'} = \mathcal{R} \cdot \prod y_{k^*}^{h'_{k^*}}$. Subtracting the two exponents yields $\hat{\pi} - \hat{\pi}' = \sum_{k^*=1}^{L^*} (h_{k^*} - h'_{k^*}) \sigma_{k^*}$ from which at least one witness σ_{k^*} can be extracted. Hence, any successful forgery would solve DLP in $\mathbb{G}$. Therefore, ProofVerify($\cdot$) is sound in the random-oracle model.

7. Performance analysis

7.1. Functionality comparison

As shown in Table 2, we compare our scheme with state-of-the-art ABKS schemes along six dimensions: dynamic data sharing, blockchain, multi-owner setting, multi-keyword search, result verification, and revocation & traceability. Most prior schemes provide only partial features, whereas BAMKS supports all six simultaneously, making it more suitable for real-world applications.

7.2. Theoretical performance

We perform a theoretical analysis of computational and storage overhead of our scheme in comparison with closely related schemes, including BLUMA [8], MKSV-ABE [21], EMK-ABSE [22], BA-RMKABSE [33], BC-dPEKS [40], and VMSE [54].

Table 3 shows that BLUMA [8] incurs the highest computational overhead in generating decryption keys. This is mainly because it maps each attribute to a fixed-length hash value via $H(\text{att}_i)_{i \in S_{\text{uid}}}$. Although BLUMA supports large-universe attributes, this benefit comes at a considerable overhead. Among comparable schemes, our scheme achieves the lowest computational cost for KeyGen($\cdot$). For notational simplicity, we use Encrypt($\cdot$) to denote the complete ABE-based access token encryption procedure, comprising both $\text{TokenEnc}_{\text{off-chain}}$ and $\text{TokenEnc}_{\text{on-chain}}$, hereafter. The computational cost of Encrypt($\cdot$) in BAMKS is higher than that in MKSV-ABE [21] and EMK-ABSE [22]. However, since we employ a hybrid off- and on-chain encryption mechanism, the output of $\text{TokenEnc}_{\text{off-chain}}$ (i.e., ICT_Φ) remains reusable even when the access policy is updated. Consequently, only the on-chain ciphertext associated with the policy requires re-computation, significantly reducing the encryption overhead. BAMKS minimizes the total decryption cost by outsourcing computationally intensive operations to the ES, leaving only lightweight final computations (i.e., E_T) for the DU.

Table 4 shows that the time overhead of IndexGen($\cdot$) grows with the number of keywords in the dictionary (i.e., $|m|$). Our scheme and MKSV-ABE [21] are the only two that support a multi-owner setting. However, MKSV-ABE incurs substantially higher keyword encryption time because its IndexGen($\cdot$) performs per-attribute computations over the access policy, so the cost increases with the number of policy attributes (i.e., $|\mathcal{L}|$). By contrast, in BAMKS the cost of IndexGen($\cdot$) scales mainly with the number of keywords, largely independent of the policy size, resulting in lower computational overhead. All schemes support multi-keyword search by generating a single trapdoor. However, in [21] and [22], the trapdoor computation is based on the user's attribute secret keys, resulting in linearly increasing TrapGen($\cdot$) time as the number of attributes grows. In contrast, our scheme aggregates the query keywords before generating $\text{Trap}_{\mathcal{W}}$, which keeps the computational overhead of TrapGen($\cdot$) lower and more stable. Regarding Search($\cdot$) time, our scheme achieves the lowest computational cost among all compared schemes: it is comparable to BA-RMKABSE [33], significantly lower than [21], and consistently more efficient than other representative schemes [22,40,54].

Table 5 shows that the storage overheads for user secret keys and ciphertexts increase with the number of attributes. Our theoretical analysis indicates that, among the compared schemes, ours yields the lowest storage cost for attribute secret keys. Although the ciphertext size in our scheme is comparatively large, we adopt a hybrid encryption architecture in which part of the encryption is performed off-chain and the remainder on-chain. The immutable ciphertext ICT_Φ is stored only once with size $2|\mathcal{L}||G| + |\mathcal{L}||G_T|$, while each policy update stores only a new on-chain ciphertext of size $(|\mathcal{L}| + 1)|G| + |G_T|$, which is smaller than those reported in [8], [22], and [33]. In addition, our scheme is space-efficient in storing the index and trapdoors, whose sizes are independent of the attributes. The size of the index $\text{Index}_{\mathcal{W}}$ scales with the keyword dictionary size $|m|$ and the number of DOs $|N|$. In terms of trapdoor size, our scheme incurs slightly higher overhead than [33], while remaining lower overhead than [22].

7.3. Experimental performance

7.3.1. Simulation environment

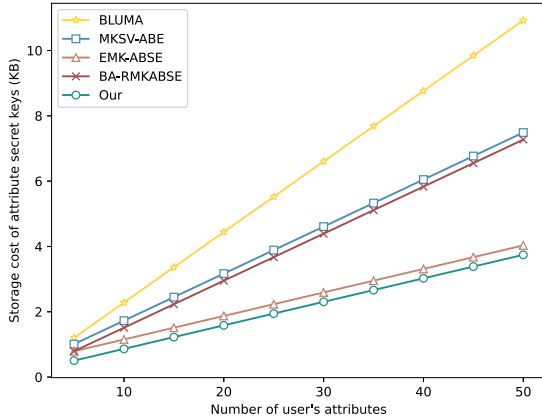
We implemented all simulations in Python 3.7 using the Charm-Crypto 0.50 and web3 on Ubuntu Linux 22.04 equipped with an Apple M1 Pro-chip and 16GB RAM. All schemes employ the SS512 pairing group and Ethereum as the blockchain platform. Experiments are conducted on the Enron Email dataset (2015)², which contains over 500,000 emails. We measure the costs of KeyGen($\cdot$), Encrypt($\cdot$), and Decrypt($\cdot$) by varying the number of system attributes from 10 to 50. Furthermore, IndexGen($\cdot$) and TrapGen($\cdot$) are evaluated on a subset of 2000 randomly selected emails with a keyword dictionary of 100 keywords ($|m| = 100$) under 50 attributes, where the number of query keywords is set to $|m^*| \in \{3, 6, 9\}$. To enable fair comparison of computational and storage overhead, all blockchain-related costs are excluded in the following evaluation.

7.3.2. Experimental computation overhead comparison

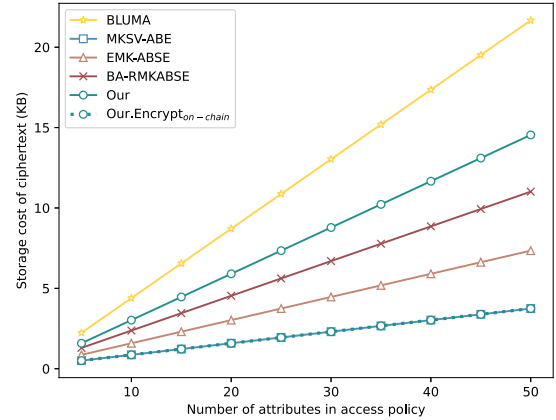
In KeyGen, Fig. 2(a) shows that the running time of all schemes grows with the number of attributes, while the growth rate of our scheme is considerably lower. When $|I|$ is 50, the time of generating user secret keys is 233.20 ms for our scheme, and 365.47 ms, 260.64 ms, 302.38 ms, and 335.91 ms for BLUMA [8], MKSV-ABE [21], EMK-ABSE [22], and BA-RMKABSE [33], respectively.

In Encrypt, Fig. 2(b) illustrates that the time cost increases roughly linearly with the number of policy attributes $|\mathcal{L}|$. In our scheme, hash operations dominate the runtime, outweighing exponentiations in G and G_T . When $|\mathcal{L}| = 50$, the total encryption time of our scheme is 119.98 ms, which is higher than that of [21] (68.26 ms) and [22] (101.49 ms). Nevertheless, the $\text{Encrypt}_{\text{on-chain}}$ time requires only 42.06 ms, indicating a significantly lower cost that is both acceptable and practical for IoT applications.

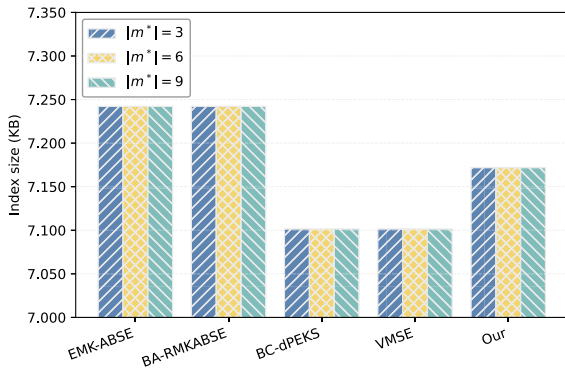
² Available at <https://www.cs.cmu.edu/~enron/>



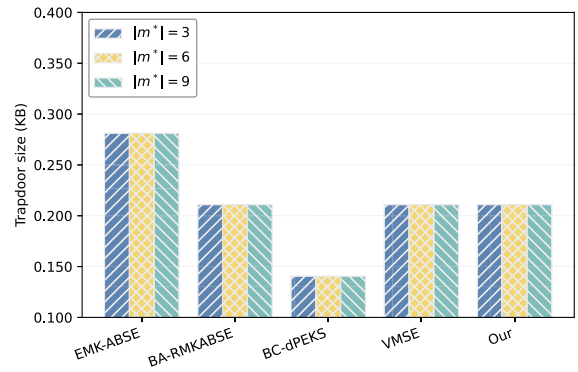
(a) Size of user secret keys



(b) Size of ciphertext

Fig. 4. Comparison storage overhead in user secret keys and ciphertext ($|m| = 100$, $|m^*| = 3$).

(a) Size of index



(b) Size of trapdoor

Fig. 5. Comparison storage overhead in index and trapdoor ($|m| = 100$, $|m^*| \in \{3, 6, 9\}$).

In Decrypt, Fig. 2(c) shows that our scheme achieves the most efficient decryption, requiring only 48.43 ms. In comparison, BLUMA [8], MKSV-ABE [21], EMK-ABSE [22], and BA-RMKABSE [33] incur 60.27 ms, 136.73 ms, 74.02 ms, and 108.96 ms, respectively.

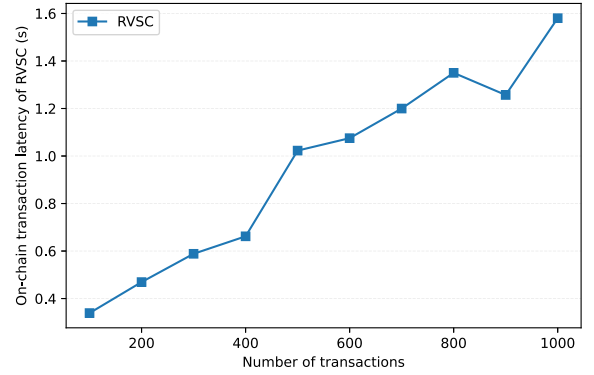
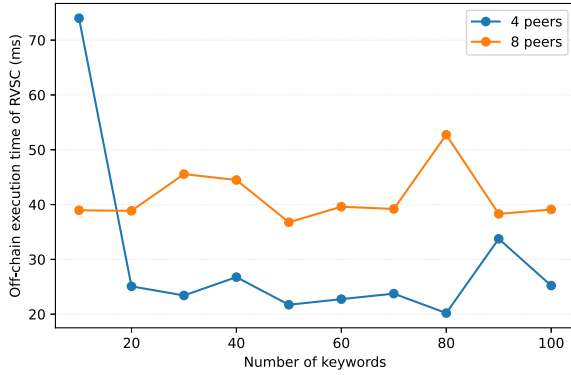
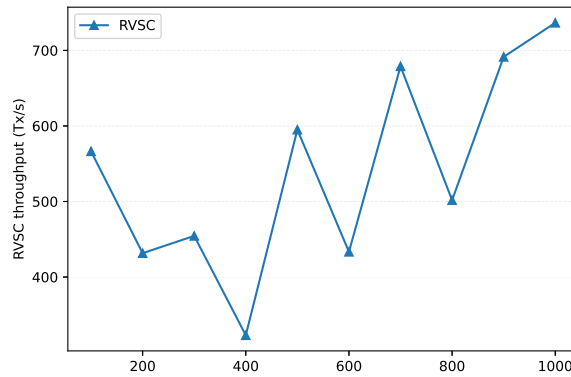
In IndexGen, Fig. 3(a) shows that the execution time of IndexGen($\cdot$) remains nearly constant with respect to the number of queried keywords $|m^*|$. When $|N| = 50$ and $|m^*| = 3$, our scheme requires only 5.11 s, which is comparable to [22], [33], and [54], but is substantially faster than [40], which incurs 27.42 s due to its more expensive pairing and multiplication operations.

In TrapGen, Fig. 3(b) indicates that trapdoor generation cost increases modestly with $|m^*|$. When $|m^*| = 3$, [22] and [40] incur considerably higher costs, averaging 66.11 ms and 10.10 ms, respectively. In contrast, our scheme maintains consistently low overhead (2.52 ms), which is close to [33] (1.95 ms) and [54] (3.12 ms). This demonstrates the efficiency of our trapdoor construction.

In Search, Fig. 3(c) shows that $|m|$ is not a critical factor in determining search time. When $|m^*| = 3$, our scheme attains competitive performance (2.28 s), comparable to [33] (2.29 s) and [54] (2.91 s), while significantly outperforming [22] (3.19 s) and especially [40] (11.27 s). These results confirm that our scheme achieves both efficiency and scalability by sustaining stable search performance irrespective of the number of attributes or queried keywords.

7.3.3. Experimental storage overhead comparison

We evaluate storage for user secret keys, ciphertexts, indexes, and trapdoors. Fig. 4(a) shows that our scheme requires only 3.656 KB for user secret keys, the smallest among the measured comparisons. Fig. 4(b) shows that our scheme initially appears to have the largest ciphertext storage requirement at approximately 14.203 KB, whereas the on-chain ciphertext storage is remarkably low at about 3.656 KB. As shown in Fig. 5(a)-(b), our scheme requires only 7.127 KB to store the encrypted keyword index when $|N| = 1$ and $|m^*| = 3$, and the index size remains constant as the number of attributes $|N|$ increases, thereby demonstrating scalability.

(a) Off-chain execution time of *RVSC*(b) On-chain transaction latency of *RVSC*(c) Throughput of *RVSC***Fig. 6.** Smart contract overhead.

In addition, the trapdoor size is maintained at just 0.211 KB. Both values are smaller than those reported in prior works, where the encrypted index reaches 7.242 KB in [22,33] and the trapdoor size is 0.281 KB in [22].

7.3.4. Experimental blockchain overhead analysis

In this section, we evaluate the performance of the *Result Verification Smart Contract (RVSC)*. To balance security and efficiency, the system adopts a hybrid on-chain/off-chain architecture, where the SNIZK proof is generated and partially verified off-chain to reduce computational overhead, while the smart contract performs the final correctness validation and records the verification result on-chain, thereby ensuring integrity and non-repudiation.

The *RVSC* was instantiated on a private Ethereum using Geth, requiring 145,448 gas for deployment and executed with a 20 % safety buffer (gas limit = 174,537, gas price = 8 Gwei). After deployment, each proof verification transaction incurred a gas fee ($\approx 21,000$ gas), as the contract enforces the final correctness validation and anchors the result on-chain. In addition, approximately 10.78 KB of data (3.656 KB of partial ciphertext and 7.127 KB of keyword index) is stored on-chain for tamper-proof auditing.

Fig. 6(a) presents the off-chain verification cost under two settings with 4 and 8 peer nodes. The experiment varies the keyword dictionary size $|m|$ from 10 to 100, while the query size ($|m^*| = 3$) and dataset (2000 emails) are fixed. Since the number of matched files remains unchanged, the proof size and the verification workload remain stable across dictionary sizes, indicating that the verification cost is determined by the query complexity rather than the dictionary size, thereby demonstrating strong scalability. When $|m| = 100$, the average verification time is approximately 25.22 ms with 4 peers and 39.10 ms with 8 peers. The moderate increase is attributed to the overhead associated with managing additional parallel threads in the local simulation environment.

Fig. 6(b) shows the transaction latency for the on-chain proof verification, measured as the end-to-end duration from transaction submission to confirmation. The experiment was conducted on a private blockchain network using a Proof-of-Authority (PoA) consensus protocol with immediate block finality, enabling consistent latency measurements. The results show that the average latency increases from 0.34 s for 100 concurrent transactions to 1.58 s for 1000 transactions. The increase results from queuing at the validator, where a larger transaction pool delays block inclusion.

Fig. 6(c) illustrates the system's throughput for the on-chain proof verification process, measured in transactions per second (TPS). The results indicate that the throughput improves as the transaction batch size increases, rising from 566.42 TPS for a batch of 100 transactions to 736.54 TPS for a batch of 1000 transactions.

In summary, although blockchain inevitably introduces additional overhead, the resulting tamper-resistance and auditable verification significantly enhance the trustworthiness of BAMKS, making it suitable for deployment in cloud-edge-IoT environments. Furthermore, BAMKS supports flexible and practical functionalities required in real-world applications.

8. Conclusion

This paper presents a blockchain-assisted, attribute-based multi-keyword search scheme for dynamic encrypted data, termed BAMKS. The scheme enables multiple data owners to collaboratively generate a token for data access, ensuring that authorized users can retrieve the version-specific files. Moreover, it supports verifiable multi-keyword search via an aggregated SNIZK proof, while providing attribute and user revocation as well as user traceability. Experimental evaluations demonstrate the efficiency and practicality of the scheme. In addition, BAMKS is proven IND-CPA and IND-CKA secure under the DBDH assumption, while outsourced decryption and proof verification are shown secure under the DLP assumption. Compared with existing works, our scheme achieves stronger security guarantees with acceptable computational and communication overheads. To the best of our knowledge, this is the first work to simultaneously satisfy all these properties, making it particularly suitable for secure and efficient cloud-edge-IoT data storage. In the future, we plan to extend BAMKS to support time-sensitive data sharing with *adding*, *deleting*, and *inserting* operations.

CRedit authorship contribution statement

Hanlei Cheng: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization; **Sio-Long Lo:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization; **Jing Lu:** Writing – review & editing, Validation, Methodology, Investigation.

Data availability

Data will be made available on request.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] D. Gupta, S. Rani, S. Raza, N.M.F. Qureshi, R.F. Mansour, M. Ragab, Security paradigm for remote health monitoring edge devices in internet of things, *J. King Saud Univ. Comput. Inf. Sci.* 35 (6) (2023) 101478.
- [2] D. Dhinakaran, N.J. Kumar, N.P. Ponnudiji, et al., Safeguarding confidentiality and privacy in cloud-enabled healthcare systems with spectrasafe encryption and dynamic k-anonymity algorithm, *Expert Syst. Appl.* 279 (2025) 127584.
- [3] V. Janarthanan, T. Annamalai, M. Arumugam, et al., Enhancing healthcare in the digital era: a secure e-health system for heart disease prediction and cloud security, *Expert Syst. Appl.* 255 (2024) 124479.
- [4] A. Sahai, B. Waters, Fuzzy identity-based encryption, in: *Advances in Cryptology—EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Aarhus, Denmark, May 22–26, 2005. Proceedings 24, Springer, 2005, pp. 457–473.
- [5] J. Camenisch, M. Kohlweiss, A. Rial, C. Sheedy, Blind and anonymous identity-based encryption and authorised private searches on public key encrypted data, in: *Public Key Cryptography—PKC 2009: 12th International Conference on Practice and Theory in Public Key Cryptography*, Irvine, CA, USA, March 18–20, 2009. Proceedings 12, Springer, 2009, pp. 196–214.
- [6] Q. Zheng, S. Xu, G. Ateniese, VABKS: verifiable attribute-based keyword search over outsourced encrypted data, in: *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*, IEEE, 2014, pp. 522–530.
- [7] Q. Huang, Q. Wei, G. Yan, L. Zou, Y. Yang, Fast and privacy-preserving attribute-based keyword search in cloud document services, *IEEE Trans. Serv. Comput.* 16 (5) (2023) 3348–3360.
- [8] H. Cheng, S.-L. Lo, J. Lu, A blockchain-enabled decentralized access control scheme using multi-authority attribute-based encryption for edge-assisted internet of things, *Internet Things* 26 (2024) 101220.
- [9] A.M.A. Alamer, A secure and privacy blockchain-based data sharing scheme in mobile edge caching system, *Expert Syst. Appl.* 237 (2024) 121572.
- [10] K. Zhang, Z. Jiang, J. Ning, X. Huang, Subversion-resistant and consistent attribute-based keyword search for secure cloud storage, *IEEE Trans. Inf. Forensics Secur.* 17 (2022) 1771–1784.
- [11] H. Gao, H. Huang, L. Xue, F. Xiao, Q. Li, Blockchain-enabled fine-grained searchable encryption with cloud-edge computing for electronic health records sharing, *IEEE Internet Things J.* 10 (20) (2023) 18414–18425.
- [12] P. Schneider, F. Xhafa, *Anomaly Detection and Complex Event Processing Over IoT Data Streams: With Application to EHealth and Patient Data Monitoring*, Academic Press, 2022.
- [13] X. Li, Q. Tong, J. Zhao, Y. Miao, S. Ma, J. Weng, J. Ma, K.-K.R. Choo, VRFMS: verifiable ranked fuzzy multi-keyword search over encrypted data, *IEEE Trans. Serv. Comput.* 16 (1) (2022) 698–710.
- [14] J. Debadarshini, M.H. Kausik, S. Saha, Structure-adaptive many-to-many data-sharing for internet-of-things, *IEEE Trans. Netw. Serv. Manage.* 21 (3) (2024) 2596–2607.
- [15] S. Rani, G. Srivastava, et al., Secure hierarchical fog computing-based architecture for industry 5.0 using an attribute-based encryption scheme, *Expert Syst. Appl.* 235 (2024) 121180.
- [16] U.S. Varri, S. Kasani, S.K. Pasupuleti, K.V. Kadambari, FELT-ABKS: fog-enabled lightweight traceable attribute-based keyword search over encrypted data, *IEEE Internet Things J.* 9 (10) (2021) 7559–7571.

- [17] U.S. Varri, D. Mallick, A.K. Das, M.S. Hossain, Y. Park, J.J. Rodrigues, TL-ABKS: traceable and lightweight attribute-based keyword search in edge-cloud assisted IoT environment, *Alexandria Eng. J.* 107 (2024) 757–769.
- [18] J. Shi, Q. Yu, Y. Yu, L. Wang, W. Zhang, Privacy protection in social applications: a ciphertext policy attribute-based encryption with keyword search, *Int. J. Intell. Syst.* 37 (12) (2022) 12152–12168.
- [19] H. Yin, Y. Li, F. Li, H. Deng, W. Zhang, K. Li, An efficient and access policy-hiding keyword search and data sharing scheme in cloud-assisted IoT, *J. Syst. Archit.* 128 (2022) 102533.
- [20] K. Gu, J. Qiu, X. Peng, X. Li, Traceable attribute-based keyword search scheme for distributed data storage under fog computing-based social internet of vehicles, *IEEE Trans. Cognit. Commun. Netw.* 11 (5) (2025) 3453–3469.
- [21] Y. Zhang, T. Zhu, R. Guo, S. Xu, H. Cui, J. Cao, Multi-keyword searchable and verifiable attribute-based encryption over cloud data, *IEEE Trans. Cloud Comput.* 11 (1) (2021) 971–983.
- [22] J. Liu, Y. Li, R. Sun, Q. Pei, N. Zhang, M. Dong, V.C.M. Leung, EMK-ABSE: efficient multikeyword attribute-based searchable encryption scheme through cloud-edge coordination, *IEEE Internet Things J.* 9 (19) (2022) 18650–18662.
- [23] Q. Huang, G. Yan, Q. Wei, Attribute-based expressive and ranked keyword search over encrypted documents in cloud computing, *IEEE Trans. Serv. Comput.* 16 (2) (2022) 957–968.
- [24] W. Li, L. Xu, Y. Wen, F. Zhang, Conjunctive multi-key searchable encryption with attribute-based access control for EHR systems, *Comput. Stand. Interfaces* 82 (2022) 103606.
- [25] L. Chen, S. Xu, C. Jin, H. Zhang, J. Weng, Partially hidden policy attribute-based multi-keyword searchable encryption with verification and revocation, *IEEE Trans. Mob. Comput.* 24 (9) (2025) 9020–9035.
- [26] M. Wang, Y. Miao, Y. Guo, C. Wang, H. Huang, X. Jia, Attribute-based encrypted search for multi-owner and multi-user model, in: *ICC 2021-IEEE International Conference on Communications*, IEEE, 2021, pp. 1–7.
- [27] H. Yin, Y. Li, H. Deng, W. Zhang, Z. Qin, K. Li, An attribute-based keyword search scheme for multiple data owners in cloud-assisted industrial internet of things, *IEEE Trans. Ind. Inf.* 19 (4) (2022) 5763–5773.
- [28] J. Li, Y. He, K. Zhang, P. Yuan, J. Yu, Fuzzy multi-keyword search for multi-owner scenario in IoV, *IEEE Trans. Veh. Technol.* 74 (4) (2024) 5698–5712.
- [29] J. Han, Y. Son, Design and implementation of a decentralized document management system, *Expert Syst. Appl.* 262 (2025) 125516.
- [30] S. Niu, M. Song, L. Fang, F. Yu, S. Han, C. Wang, Keyword search over encrypted cloud data based on blockchain in smart medical applications, *Comput. Commun.* 192 (2022) 33–47.
- [31] J. Sun, W. Su, L. Wang, K. Ye, Multi-keyword ranked search scheme for privacy protection in social networks, *Comput. Netw.* 249 (2024) 110534.
- [32] X. Xiang, X. Zhao, Blockchain-assisted searchable attribute-based encryption for e-health systems, *J. Syst. Archit.* 124 (2022) 102417.
- [33] J. Su, L. Zhang, Y. Mu, BA-RMKABSE: blockchain-aided ranked multi-keyword attribute-based searchable encryption with hiding policy for smart health system, *Future Gener. Comput. Syst.* 132 (2022) 299–309.
- [34] V. Bakhtiary, M. Mirabi, A. Salajegheh, S.H. Erfani, Combo-chain: towards a hierarchical attribute-based access control system for IoT with smart contract and sharding technique, *Internet Things* 25 (2024) 101080.
- [35] D. Zhang, S. Wang, Q. Zhang, Y. Zhang, Attribute based conjunctive keywords search with verifiability and fair payment using blockchain, *IEEE Trans. Serv. Comput.* 16 (6) (2023) 4168–4182.
- [36] J. Wang, X. Lin, Y. Wu, J. Wu, Blockchain-enabled lightweight fine-grained searchable knowledge sharing for intelligent IoT, *IEEE Internet Things J.* 10 (24) (2023) 21566–21579.
- [37] K. Zhang, Y. Zhang, Y. Li, X. Liu, L. Lu, A blockchain-based anonymous attribute-based searchable encryption scheme for data sharing, *IEEE Internet Things J.* 11 (1) (2023) 1685–1697.
- [38] H. Yin, Y. Zhao, L. Zhang, B. Qiao, W. Chen, H. Wang, Attribute-based searchable encryption with decentralized key management for healthcare data sharing, *J. Syst. Archit.* 148 (2024) 103081.
- [39] F. Shen, L. Shi, J. Zhang, C. Xu, Y. Chen, Y. He, BMSE: blockchain-based multi-keyword searchable encryption for electronic medical records, *Comput. Stand. Interfaces* 89 (2024) 103824.
- [40] H. Yu, S. Liu, L. Chen, Y. Gao, Blockchain-enabled one-to-many searchable encryption supporting designated server and multi-keywords for Cloud-IoMT, *J. Syst. Archit.* 149 (2024) 103103.
- [41] Z. Yan, B. Zhang, An efficient attribute-based multi-keyword searchable encryption with access policy hiding in IoT using blockchain, *IEEE Internet Things J.* 12 (15) (2025) 32148–32160.
- [42] R. Hou, F. Zhou, Q. Wang, Z. Jiao, J. Sun, Z. Zhang, Revokable blockchain-enabled ranked multi-keyword attribute-based searchable encryption scheme with mobile edge computing for vehicular, *IEEE Trans. Netw. Serv. Manage.* 22 (3) (2025) 2764–2779.
- [43] S. Nakamoto, Bitcoin: a peer-to-peer electronic cash system, *Decentralized Bus. Rev.* (2008).
- [44] M. Iqbal, S. Suhail, R. Matulevičius, F.A. Shah, S.U.R. Malik, K. McLaughlin, IoV-TwinChain: predictive maintenance of vehicles in internet of vehicles through digital twin and blockchain, *Internet Things* 30 (2025) 101514.
- [45] T.L. Nguyen, L. Nguyen, T. Hoang, D. Bandara, Q. Wang, Q. Lu, X. Xu, L. Zhu, S. Chen, Blockchain-empowered trustworthy data sharing: fundamentals, applications, and challenges, *ACM Comput. Surv.* 57 (8) (2025) 1–36.
- [46] H. Zhu, L. Yang, L. Wang, V.S. Sheng, A survey on security analysis methods of smart contracts, *IEEE Trans. Serv. Comput.* 17 (6) (2024) 4522–4539.
- [47] R. Song, B. Xiao, Y. Song, S. Guo, Y. Yang, A survey of blockchain-based schemes for data sharing and exchange, *IEEE Trans. Big Data* 9 (6) (2023) 1477–1495.
- [48] S. Pal, A. Dorri, R. Jurdak, Blockchain for IoT access control: recent trends and future research directions, *J. Netw. Comput. Appl.* 203 (2022) 103371.
- [49] B. Waters, Ciphertext-policy attribute-based encryption: an expressive, efficient, and provably secure realization, in: *International Workshop on Public Key Cryptography*, Springer, 2011, pp. 53–70.
- [50] Y. Rouselakis, B. Waters, Efficient statically-secure large-universe multi-authority attribute-based encryption, in: *International Conference on Financial Cryptography and Data Security*, Springer, 2015, pp. 315–332.
- [51] X. Wu, X. Du, Q. Yang, N. Wang, W. Wang, Redactable consortium blockchain based on verifiable distributed chameleon hash functions, *J. Parallel Distrib. Comput.* 183 (2024) 104777.
- [52] C. Liu, C. Hou, T. Jiang, J. Ning, H. Qiao, Y. Wu, FACOS: enabling privacy protection through fine-grained access control with on-chain and off-chain system, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 7060–7074.
- [53] A. Penuelas-Angulo, C. Feregrino-Urbe, M. Morales-Sandoval, Revocation in attribute-based encryption for fog-enabled internet of things: a systematic survey, *Internet Things* 23 (2023) 100827.
- [54] Y. Liang, Y. Li, K. Zhang, Z. Wu, VMSE: verifiable multi-keyword searchable encryption in multi-user setting supporting keywords updating, *J. Inf. Secur. Appl.* 76 (2023) 103518.